systems

MDPI

Article

A Reference Architecture for Smart Car Parking Management Systems

Mert Ozkaya [1,*] and Alper Turunc [2]

[1] Computer Engineering Department, Yeditepe University, Istanbul 34755, Türkiye
[2] DFDS, Istanbul 34746, Türkiye; turualp@dfds.com
* Correspondence: mozkaya@cse.yeditepe.edu.tr

Abstract: Smart car parking management systems (SPMSs) have gained an ever-increasing popularity for the digital management of car parking processes. While various techniques and technologies have been proposed for SPMSs, the literature lacks in any generic software architecture design that can be reused systematically for the specifications of quality SPMS architectures. To bridge this gap, we propose a reference architecture (RA) for the SPMS product family after performing a comprehensive domain analysis. Our RA design offers a feature model that consists of the common and varying features for SPMSs. We offer multiple viewpoints for our RA, including context, module, component and connector, and allocation. The context viewpoint focuses on the stakeholders, the module viewpoint focuses on the software units, the component and connector viewpoint focuses on the layered architecture of SPMSs, and the allocation viewpoint focuses on mapping software units into the physical components. Each viewpoint can be re-used for specifying the application architecture of any SPMSs. We validated our RA with a real SPMS scenario specification and prototype development, where the former measures the reusability of RA and the latter measures the development performance. The RA design for SPMSs is expected to be useful for several stakeholders who research, develop, and sell SPMS solutions.

Keywords: smart car parking; reference architecture; software architecture; multiple viewpoints

check for updates

Academic Editor: Mahyar Amirgholy

Received: 10 December 2024
Revised: 4 January 2025
Accepted: 14 January 2025
Published: 21 January 2025

Citation: Ozkaya, M.; Turunc, A. A Reference Architecture for Smart Car Parking Management Systems. *Systems* **2025**, *13*, 70. https://doi.org/10.3390/systems13020070

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The world population is expected to exceed nine billion by 2040 and several mega-cities around the world have already reached 10+ million populations [1]. Crowded cities commonly suffer from traffic congestion, which causes many issues, such as stressful life for drivers, air pollution, unproductive use of time, and traffic accidents [2]. While a variety of local transportation options are offered in mega-cities to reduce traffic congestion, most people use their own cars. Indeed, more than 1.4 billion cars have been recorded in the world and this is expected to be doubled by 2040 [3]. One of the most crucial problems contributing to traffic congestion is drivers who are looking for parking spaces. A recent survey [4] revealed that drivers spend on average 14 min to find a suitable parking space on each trip. In another study conducted by INRIX [5], UK drivers were observed to spend 44 h a year finding an available parking space. In the USA, drivers spend 17 h a year finding available parking spaces [6]. This essentially indicates not only the unproductive use of the drivers' time but also excessive fuel consumption, which causes air pollution and a redundant use of the limited resources. It should also be noted that 30% of the traffic congestion results from drivers who are looking for available parking spaces [7].

To mitigate the issues around finding available parking places, the processes of car parking management should be digitalized using smart techniques and technologies [8].

Many smart car parking management systems (SPMSs) have been developed thus far, through which various facilities can be performed, including online prior reservation, online payments, the automated detection of the closest/convenient car parking spaces, automated navigation for available parking spaces, and the integration with relevant smart applications such as fuel payment and car charging applications. As such, optimum decisions for car parking can be made quickly, manual tasks can be automated (e.g., payment and reservation), and thus, the stakeholders who are involved in the car parking processes (e.g., drivers and parking controllers) can reach their goals and use their time more productively and effectively. Moreover, this will reduce carbon emissions and traffic congestion, which in turn reduces the number of accidents and stressful drivers.

SPMSs are essentially complex systems [9] that are used by different stakeholders (e.g., drivers, park controllers, municipalities, etc.) for various purposes and are integrated with various external systems (e.g., payment systems, data analytics systems, sensors, car charging systems, etc.). Also, SPMSs consist of several different applications (e.g., navigation, reservation, street map, parking lot designer, etc.), each of which needs to be developed in terms of several interdependent software modules. Therefore, it is highly crucial to adopt a software architecture-centric perspective for the analysis, design and development of SPMSs. Software architecture is considered as a crucial technique for managing the complexity of software systems, and it focuses on decomposing software systems into components, mapping those software components into the physical components, detailing the interactions among the components, and making design decisions for meeting the quality requirements that need be satisfied [10].

To facilitate the architectural design of software systems, the notion of "reference architecture (RA)" has been proposed [11], which promotes the generic design of software architectures for a product family in a problem domain in a way that can be re-used (and extended) for designing the architectures of particular products in that product family. An RA design for a product family addresses the mandatory and variable product features and it provides a set of generic, reusable design artifacts that enable the establishment of boundaries of a particular product and specifying the principle design decisions from different architectural perspectives (e.g., logical, physical, and development) for that product. Using RA, technical and non-technical stakeholders who are involved in the problem domain can easily communicate regarding the needs of any product of the product family in question, and thus, high-quality products that better meet the requirements can be analyzed, designed and developed with the least effort and smallest budget possible.

Given the crucial nature of car parking management, many different techniques and technologies can be used for digitalizing car parking. Indeed, the literature includes several studies on the design and development of SPMSs. As discussed in Section 2, the existing studies either *(i)* introduce specific SPMS applications, *(ii)* discuss the applications of particular data analytics, information or communication technologies for SPMSs, or *(iii)* discuss the analysis of the existing SPMS applications for some features of interest. However, none of the existing studies considered proposing a generic solution that can aid in analyzing, designing and implementing quality SPMS products with the least cost possible. Indeed, despite the crucial nature of SPMSs for society and the ever-increasing trend in developing SPMSs, no RA design has yet been proposed for the SPMS product family. Therefore, lacking any generic solutions such as an RA design prevents understanding of the common and varying features for any SPMS product, the possible stakeholders who could be involved in the product life cycle of SPMSs, and most importantly, the optimum partitioning of SPMS solutions with regard to different concerns (e.g., logical, physical, and development). As such, without the RA design for SPMSs, the SPMS community is likely to waste their efforts in the development processes of SPMS products. That is, the requirements for any specific

SPMS product may not be gathered and analyzed in a precise manner and the crucial steps such as the stakeholder analysis and architectural design can even be omitted. As revealed via our case study evaluations in Section 6, developing SPMS products without an RA design may even lead to project failures.

In this paper, the goal is to design an RA for the SPMS product family that can be re-used for designing application-specific SPMS architectures. Firstly, a feature model is specified, which indicates the common and varying functionalities of SPMSs and their quality characteristics. Then, multiple architectural viewpoints are specified for managing the complexity of SPMSs. These viewpoints are *(i)* context, *(ii)* module, *(iii)* component and connector, and *(iv)* allocation. Our RA design for SPMSs is believed to be useful for different stakeholders who are involved in car parking management processes. The companies who offer SPMS solutions can identify the mandatory and optional features for SPMSs and improve their tools. Software developers (or architects) who design and develop SPMS solutions can use our RA viewpoints and easily specify the application architectures (AAs) for the specific SPMS products of their interest. Indeed, the RA viewpoints can be extended and adapted to the specific product needs by adding/removing new/existing elements. Lastly, researchers can use our RA and understand the requirements and design expectations for SPMSs. So, researchers can initiate research projects that propose innovative techniques and technologies for car parking management.

2. Related Work

The literature includes tens of different studies about smart car parking, which shed light on different concerns. Some of those studies propose the applications of different data source technologies and software engineering techniques and technologies. These include the approaches that *(i)* apply dynamic resource allocation algorithms [12], *(ii)* use cloud computing [13], *(iii)* use ultrasonic sensors for detecting empty parking spaces [14], *(iv)* use convolutional neural network for detecting empty spaces [15], *(v)* use genetic algorithm for car scheduling [16], *(vi)* use fog computing for designing the architectures of smart parking systems [17], and *(vii)* use RFID technology [18].

The literature also includes different survey studies that aid in comparing the existing car parking solutions from diverse perspectives. Fahim et al. [19] compared a number of SPMSs for a set of features which include software technologies (e.g., AI technologies), the sensor devices, different computational approaches, networking technologies, and user interface types. Paidi et al. [20] analyzed different sensor devices for collecting parking data and compared a set of SPMSs for different sensor devices. Lin et al. [8] performed a comprehensive analysis of a set of SPMSs for a large feature set. Lin et al.'s feature set is mainly categorized into three groups, which are information collection, system deployment, and service dissemination, and each group is further divided into sub-groups. Xiao et al. [21] focused on the car parking prediction problem for finding an empty parking space. Xiao et al. compared different prediction methods in terms of their strengths and weaknesses, analyzed the prediction problems, and further suggested research ideas on tackling the challenges. Barriga et al. [22] analyzed a huge set of papers that discuss smart parking solutions for three main features. These features are the sensor type, the network infrastructure, and the services that the smart parking solutions offer (e.g., information management, e-parking, and parking space prediction).

The literature includes some other studies that discuss the architecture design of SPMS solutions. Fraifer et al. proposed a prototype-centric design and development of SPMS solutions and focus on understanding the user needs [23]. Anderson et al. discuss the architecture for a specific SPMS based on fog computing [24]. Anderson et al. focus on the vehicle detection and proposed a set of component types that can be used for designing the

SPMS architectures in a way which considers the secure and fast communication. Soegoto et al. discuss the hardware and software units that can be composed for an SPMS [25]. Soegoto et al. focus on car detection and developed a prototype on their solutions. Awaisi et al. discuss an architecture framework for SPMSs which is based on fog computing [26]. Awaisi et al. proposed a layered architecture that consists of three layers. The bottom layer consists of camera devices (as the data sources), the middle layer consists of fog nodes that can collect data from the camera devices and process those data for detecting the parking space occupancies, and the top layer includes a cloud server for storing the processed data. Alharbi et al. proposed a web-based framework for smart car parking [27]. Alharbi et al. discuss an algorithm for detecting car plates through which the car parking gates can be opened/closed. Alharbi et al. also discuss their web-based tool implementation, which provides such services as performing booking for any parking spaces, managing visitor data, and generating useful reports.

While the literature includes diverse studies on SPMSs, none of the existing studies provide a generic solution that can be re-used systematically for the analysis, design and implementation of SPMSs. It should be noted that many RAs have been proposed for different domains and industries, such as health center information systems [28], smart farming [29], autonomous driving [30], digital twins [31]. Considering the literature's lack of support for an RA design in the SPMS domain, we consider that this is a crucial issue which is worth dealing with. Indeed, SPMSs are essentially complex systems with diverse features and need to be analyzed and designed systematically using well-accepted practices. Otherwise, practitioners may end up with any important features that have been skipped, wrong (i.e., ineffective) design decisions, or unmanageable code that cannot easily be maintained.

3. Research Questions

In our RA design for the SPMS product family, we aim to investigate four key research questions (RQs) that are discussed as follows.

RQ1: Is it possible to analyze the smart car parking domain so as to determine the common and varying features? Our RA design in this paper focuses on the family of SPMS products rather than a single, specific product. So, we initially identified all possible features that any SPMS products may possess and categorized those features as the common features and optional features for SPMSs. Indeed, while some of the functionalities and quality concerns are expected to be supported by each SPMS product, the existence of some features depends on the problem domains in which SPMSs are used. In Section 4, RQ1 is addressed and the feature model of the SPMS product family is discussed.

RQ2: Is it possible to determine the stakeholders who interact with SPMSs and their responsibilities? The needs of stakeholders for any software system can influence the quality of software systems, and thus, considering those needs aids in better managing the system requirements [32]. Therefore, upon determining the feature model for SPMSs (RQ1), we identified the stakeholders who are involved in the SPMS development and operation phases and the stakeholders' roles and responsibilities in the SPMS context. RQ2 is addressed in Section 5.2, where the system boundaries for SPMSs are discussed in terms of the stakeholders and their system interactions.

RQ3: Is it possible to design an RA for SPMSs in a manageable way for the facilitated understandability and communication? An RA design needs to manageable so that it can easily be understood and communicated for the AA specifications of any specific SPMS solutions. As indicated by several important books on software architectures [33,34], the complexity in software architecture design can best be handled with the multiple-viewpoints approach. Therefore, we proposed our RA design for the SPMS product family in terms multiple

viewpoints, each of which deals with a separate concern for SPMS development and describes a generic architectural solution for that concern. RQ3 is addressed in Section 5, where we introduce our RA design in terms of the *(i)* context, *(ii)* module, *(iii)* component and connector, and *(iv)* allocation viewpoints.

RQ4: Is it possible to validate our RA design for SPMSs for the development quality properties? To validate our RA design for SPMSs, we focus on the development quality properties rather than the run-time quality properties (e.g., security, availitabily, and usability) [35]. So, we aimed to measure our RA's support in reducing the development cost, and therefore, we considered two main properties, which are the *(i)* design re-usability and *(ii)* development performance. With reusability, the goal is to understand to what extent our RA design can be re-used for the specifications of AAs. To this end, we focus on a scenario-based evaluation on a commercial SPMS product and measuring the ratio of the reused elements from our RA design in the AA specification of the SPMS product. With the development performance, the goal is to understand to what extent our RA design aids in improving the development performance of SPMS products in terms of the time spent for the analysis, design, implementation and testing stages of SPMS development. To this end, we focus on a prototype evaluation and measuring the development performance while developing an SPMS prototype. That is, we worked with two separate groups of software developers where one group developed an SPMS product without our RA and another group developed an SPMS product with the use of our RA. By doing so, we were able to measure the time that each group spent on different phases of the development life cycle and compare the time spent by the groups for each development phase. In Section 6, the scenario-based and prototype evaluations are discussed.

To address the research questions given in Section 3, we followed the research method that is illustrated in Figure 1. Firstly, we performed a domain analysis, which is discussed in Section 4. Domain analysis is concerned with determining the SPMS features by means of a comprehensive literature review study and practitioner survey. After determining the features, we determined the stakeholders for the SPMS domain.

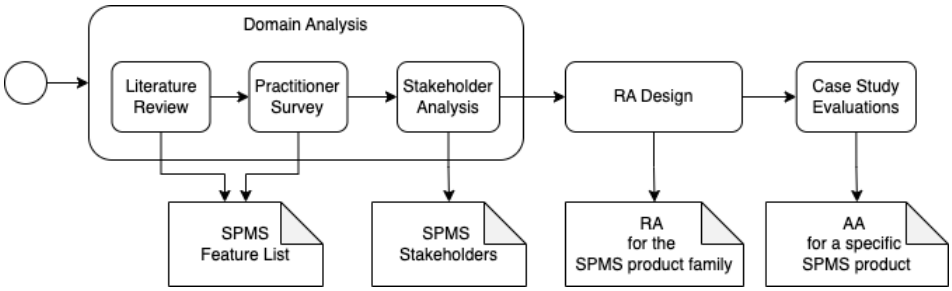

Figure 1. The research method for the RA design.

After performing the domain analysis, we proposed our RA design for the SPMS product family in terms of multiple viewpoints, as discussed in Section 5.

Lastly, we performed the case study evaluations, as discussed in Section 6, which involved deriving AAs from our RA design and performing the development quality measurements.

4. Domain Analysis for SPMSs

To understand the problems on the car parking management, we aimed at obtaining the car parking domain knowledge in a systematic way. To this end, we performed a comprehensive literature review and collected a great list of papers about smart car parking. The papers that we considered here either *(i)* analyze the existing parking management tools for some relevant innovative techniques and technologies, e.g., [8,19,22], *(ii)* introduce smart

parking management solutions, e.g., [36–38], or *(iii)* discuss the applications of some crucial techniques and technologies, e.g., [39–41]. We went through each paper carefully and determined the functionalities, quality characteristics, and different types of stakeholders that are discussed in the papers. Furthermore, we searched via *Google* and found many commercial SPMSs, such as Fleximodo[1], Parkable[2], and 4Park[3]. For each commercial SPMS, we went through their tool web-site and determined any knowledge resources (e.g., videos, white papers, case studies, and user guides, feature comparison tables) so as to find the tool functionalities, the supported quality properties, and any involved stakeholders who can use the tools via some user interfaces. So, our domain analysis of the literature and commercial SPMS products allowed for identifying *(i)* the potential stakeholders for SPMSs, their roles and responsibilities, and *(ii)* the functionalities and quality properties that can be considered for the SPMS solutions and their levels of importance.

To analyze the SPMS functionalities, quality concerns, and stakeholders that we determined via our domain analysis, we further conducted a face-to-face survey with a group of parking controllers. The parking controllers here work for ISPARK (Istanbul Parking Lot Operations Trade S.A)[4], which is a subsidiary for the municipality of Istanbul, Turkey and responsible for operating the car parking services in Istanbul. We observed that the parking controllers do not currently use any mobile applications. The parking controllers direct the drivers who drive down their streets towards empty parking places manually and charge the drivers depending on their estimated time of departure. The parking controllers are responsible for the manual detection of any drivers who exceeded their stated departure time and charging the drivers extra fees. Our survey includes 15 questions and has been completed by 15 different parking controllers. In our survey, we essentially aimed to understand *(i)* the challenges that the parking controllers face while operating the car parking services without the use of any SPMSs and *(ii)* the controllers' expectations from an SPMS[5]. After analyzing the survey results, we were able to understand *(i)* the importance of smart technologies for drivers and parking controllers in managing the car parking processes, *(ii)* the mandatory and optional functionalities and quality properties for drivers and parking controllers, and *(iii)* the most important stakeholders for car parking management.

After collecting and analyzing the SPMS domain and determining the features, we focused on identifying the stakeholders and their responsibilities. For each feature that we determined, we identified any stakeholders who interact with SPMSs and cause that feature to be enabled. We went through all the resources that have been collected and analyzed the features (e.g., tool web-sites, documents, papers, and interview results) and identified *(i)* the different types of users and user groups for the SPMS solutions, *(ii)* software and physical systems that can integrate into SPMSs, *(iii)* any types of data sources that can provide data for SPMSs, and *(iv)* any government institutions and law enforcement agencies that require data from SPMSs. We further validated the identified list of stakeholders via our interviews with the ISPARK staffs. Table 1 gives our final list of stakeholders for the SPMS product family. The driver acts as the main user who interacts with an SPMS so as to perform the driver-related tasks, e.g., finding an empty parking space, booking, and navigating to the empty parking spaces. The parking controller interacts with an SPMS so as to manage the parking operations, e.g., parking reservations and payments. The municipality staff and parking provider interact with an SPMS so as to perform the administration tasks such as monitoring, generating and viewing useful reports, budget management, tariff management, billing and payment management, etc. The software developers are responsible for designing and developing an SPMS with regard to the system requirements and managing the software testing processes. The system administrator is concerned with administering an SPMS and performing the maintenance tasks such as ensuring the use

of up-to-date technologies and installing and maintaining new software/hardware. The data sources (e.g., sensor and camera) acquire data from the environment of an SPMS such as the sensor data for detecting parking space occupancies. Government systems and any other external systems can be integrated into SPMSs so as to provide/require such services as parking fine management, fuel management, and charging station management.

Table 1. The stakeholders involved in SPMSs and their goals.

Stakeholder	Goal
Driver	To use SPMS and perform driver-related tasks
Parking Controller	To use SPMS and control the car parking processes
Municipality	To use SPMS and get informed about the car parking processes
Parking Provider	To use SPMS and get informed about the car parking processes
Software Developer	To design, develop and test SPMS
System Administrator	To administer and maintain SPMS
Data Scientist	To analyse the data maintained by SPMS
Sensor Data Source	To provide data about the parking place occupancy
Camera Data Source	To provide data about the parking place occupancy
Government	To integrate government e-services and require/provide data
External System	To integrate with the car parking management systems

To model the functionalities and quality properties that we determined via our SPMS domain analysis, we used FODA's feature modeling methodology [42]. Feature modeling promotes the specifications of mandatory and optional features that represent the user-visible functionalities and quality characteristics for a product family. So, a feature model for a product family indicates *(i)* the features that any product of that family must possess and *(ii)* the features that are optional and thus do not always have to be supported. Feature models are specified in a hierarchical way where any features can be described in terms of its sub-features. Note that sub-features can be the optional or mandatory part of the features. A feature may have an "OR" relationship with its sub-features, which indicates that the feature needs to possess at least one of its sub-features. Figure 2 shows the feature model that we specified for the SPMS product family. The mandatory features that are expected to be provided by any SPMSs are parking lot, parking map, security and data management. The parking lot feature serves for designing and editing the car parking layout, which is concerned with such functionalities as *(i)* editing the car entrance/exit directions, roads and any accessible parking areas, *(ii)* locating the parking spaces, and *(iii)* specifying other physical objects (e.g., buildings, trees, etc.). The parking lot feature can also enable viewing the parking lot for determining parking space occupancies, the optimized use of the car parking area, and any reporting facilities. The parking map feature serves for displaying the closest parking areas on a visual map and provides many functionalities such as an offline parking map, an online parking map that shows the parking occupancies in real time, exporting the selected parking areas as an image file, saving any parking locations, editing any parking locations (e.g., adding photos), sharing parking locations on social media, and facility management (e.g., viewing the nearest hotels, restaurants and cafes). Note that the online parking map and real-time parking lot occupancy monitoring functionalities are the mandatory sub-features for the parking map feature. The driver feature provides several functionalities for drivers, including *(i)* viewing any useful reports about their system usages, *(ii)* managing the notifications, preferences, payment methods, and memberships, *(iii)* viewing the parking history information, and *(iv)* help and support. The security feature is for securing the system data and can be considered in terms of the important principles of information security (also called "CIA triad"), which are confidentiality, integration, and

availability. Note that the security feature must possess at least one of its sub-features. The data management feature serves for collecting data from data sources (e.g., sensors, camera, and user interfaces), storing the collected data persistently/temporarily, processing the collected data, making any decisions using the data analytics techniques (e.g., machine learning), and visualizing those decisions. Just like security, the data management feature must possess at least one of its sub-features.

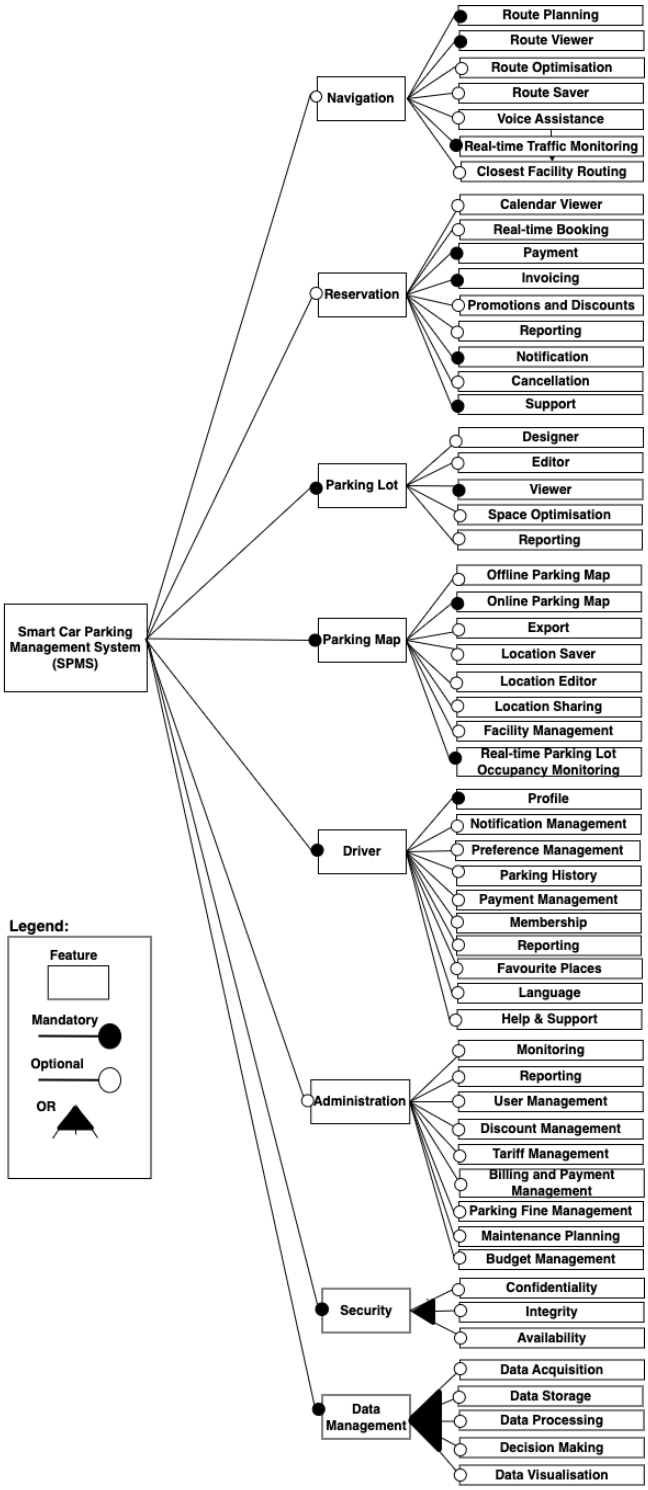

Figure 2. The feature model for SPMS.

Besides the mandatory SPMS features, Figure 2 shows various features that are optional. These are navigation, reservation, user profile, and administration. The navigation

feature serves for determining the optimal route from the driver's current location to the selected parking place and can further offer such functionalities as route saving, voice assistance, and closest facility routing. The reservation feature serves for reserving an available car parking place for a particular time slot, making payment, and other functionalities such as cancellation, invoicing, applying discounts, and receiving notifications. Lastly, the administration feature serves for administering an SPMS and provides several functionalities for administrators, which include *(i)* system monitoring, *(ii)* generation of reports about, e.g., users, payments, parking areas, etc., *(iii)* user management (e.g., adding/removing users), *(iv)* managing billing and payment, discounts, tariffs, parking fines, and budget, and *(v)* planing and scheduling the system maintenance.

5. Reference Architecture (RA) Design for SPMS

The architectural design for any software systems is concerned with making high-level design decisions about different concerns of the involved stakeholders (e.g., hardware designer, software designer, and programmer). To manage the complexity of software architectures, multiple-viewpoints modeling can be adopted [33]. Here, each viewpoint addresses a particular concern (e.g., logical units, physical architectures, and development units) and proposes *(i)* the division of systems into manageable parts and *(ii)* any rules and constraints on the system divisions and their parts so as to achieve the stakeholders' goals.

5.1. Selection of Viewpoints

To design an RA for the SPMS product family, we considered the viewpoints that are encouraged by Clements et al.'s seminal book on the architecture viewpoints [34] and commonly accepted by the software architecture community. Clements et al.'s approach is essentially based on the pioneering viewpoint models of Kruchten's 4+1 [43] and Siemens Four Views [44]. According to Clements et al., software architects must always consider the three main aspects of software architecture design, which are the system's *(i)* logical structure, *(ii)* behaviors and interactions, and *(iii)* physical structure. Therefore, we propose four main viewpoints in this paper, which are *(i)* context, *(ii)* module, *(iii)* component and connector, and *(iv)* allocation. The context viewpoint is concerned with *(i)* establishing the system boundaries, *(ii)* any stakeholders and their communications with the system, and *(iii)* the integration with any external systems. The module viewpoint is concerned with the systems' decomposition into the implementation units where each unit is supposed to realize a cohesive set of responsibilities (e.g., functionalities and quality expectations). The component and connector viewpoint is concerned with systems' runtime behaviors, where the components represent the systems' running units and connectors represent the protocols of interactions among the components. To organize the components and connectors, we considered here the layered architecture style. Indeed, we strongly believe that the layered style fits very well with such types of information systems as SPMSs where the components are typically separated into the distinct layers of the user interaction, business logic and data. Lastly, the allocation viewpoint is concerned with mapping the software components into the physical devices that are physically connected.

Besides Clements et al.'s approach, many other architecture viewpoint frameworks have been proposed. For instance, Rozanski et al.'s approach [33] offers six different architecture viewpoints, which are functional, information, concurrency, development, deployment, and operational. Garland et al. [45] offer fourteen different viewpoints in their framework, which include such viewpoints as analysis, component, interaction, context, deployment, layered subsystem, logical data, physical data, process, and subsystem interface. While those viewpoint frameworks could also be preferred, Clements et al.'s approach is highly popular for the specification of RAs, as many RA designs on diverse

problem domains adapt Clements et al.'s approach [28,29,46–48]. Therefore, Clements et al.'s approach can be considered as a standard architecture style for designing RAs, and this can enhance the understandability of RA designs which are based on the same standard architecture style. Note also that Clements et al. consider the essential viewpoints for the architecture design, while other frameworks offer a greater number of viewpoints and reduce the level of abstraction. Indeed, RAs that impose a number of viewpoints may hinder the re-use of RAs for the specifications of AAs. As demonstrated via our case study evaluations that are discussed in Section 6.1, we successfully managed to reuse our RA viewpoints in the AA specification of a real SPMS.

5.2. Context Viewpoint

The context viewpoint is depicted in Figure 3, which shows the stakeholders that can interact with an SPMS. Two types of stakeholders are considered here, which are the mandatory and optional stakeholders. The mandatory stakeholders must exist for each SPMS, who can be either software developers, administrators or drivers. The optional stakeholders are not essential and their existence depends on the requirements of an SPMS to be developed. The optional stakeholders can be either data scientists, park attendants, municipality staff, or research institutes. Note that the optional stakeholders can also represent any external systems that an SPMS can be integrated with, such as data sources (sensors and camera), e-government systems, and electric car charging systems.

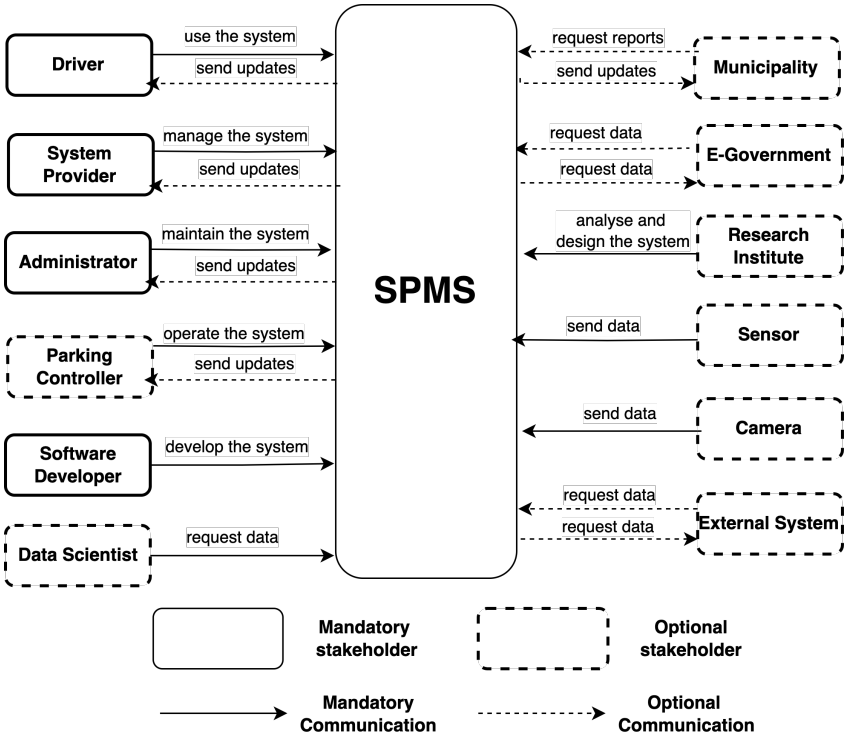

Figure 3. The context viewpoint for SPMS.

Stakeholders and an SPMS can exchange data or control among each other through the communication links. As depicted in Figure 3, communication links can be either optional or mandatory. Indeed, not all SPMSs have to send updates to the system provider or administrator, while each administrator is expected to maintain the system. Also, the communication among a stakeholder and an SPMS can be performed in either one way or two ways (i.e., request and response). For instance, an SPMS is expected to establish a two-way optional communication link with a municipality. An SPMS typically establishes a mandatory communication link with a driver, as the driver will need to use the system.

However, drivers receive updates from an SPMS optionally, which is the case only if an SPMS has such a requirement.

5.3. Module Viewpoint

The module viewpoint for our RA design is depicted in Figure 4, which shows the implementation units that can be designed and developed for an SPMS. Here, we considered the feature model that is given in Figure 2 and proposed eight different main units, each of which consists of development modules (and sub-modules). Note that the software modules herein represent the main units of implementation, which can be implemented as a software package, class, or even a function.

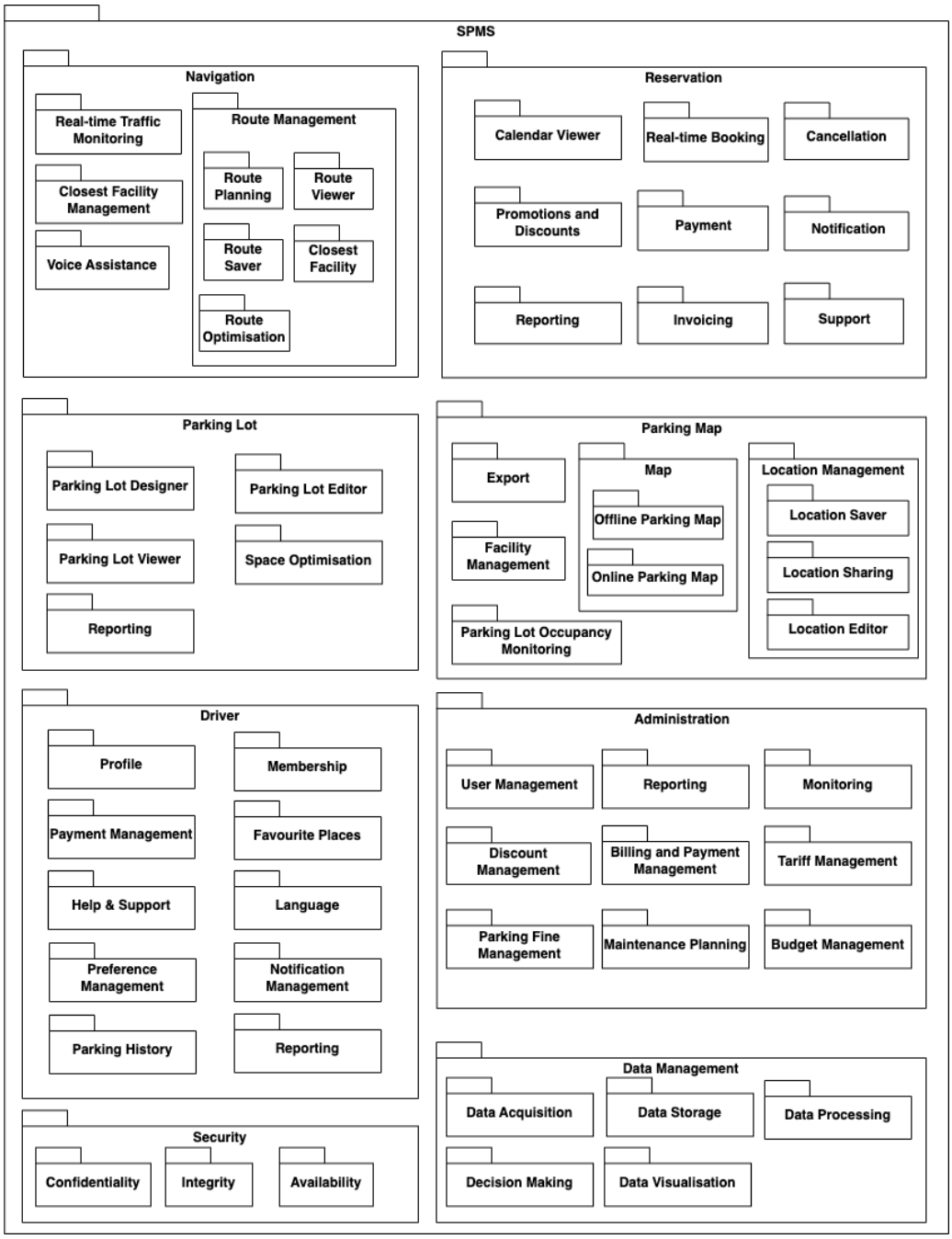

Figure 4. The module viewpoint for SPMS.

The navigation unit represents the SPMS functionalities that can be used by drivers and is concerned with navigating to the available car parking spaces safely. The navigation

unit consists of four main modules, which are *(i)* voice assistance, *(ii)* real-time traffic monitoring, *(iii)* closest facility management, and *(iv)* route management. The route management module here is further decomposed into a cohesive set of sub-modules, which are *(i)* route planning, *(ii)* route viewer, *(iii)* route searcher, *(iv)* route saver, *(v)* route optimization, and *(vi)* routing for the closest facility. The reservation unit represents the SPMS functionalities that can be used by drivers and parking controllers and is concerned with reserving an empty car parking place. The reservation unit consists of several modules, which are *(i)* calendar viewer, *(ii)* real-time booking, *(iii)* payment, *(iv)* promotions and discounts, *(v)* cancellation, *(vi)* notification, *(vii)* invoicing, *(viii)* reporting and *(ix)* support. The parking lot unit represents the SPMS functionalities that are used by parking controllers and system providers and is concerned with the parking area layout for managing the available parking lots. The parking lot unit encapsulates such modules as *(i)* parking lot designer, *(ii)* editor, *(iii)* lot viewer, *(iv)* optimization for using the parking space effectively, and *(v)* reporting. The parking map unit represents the SPMS functionalities that can be used by drivers and parking controllers and is concerned with the parking map that shows the closest parking areas. For instance, a street map shows the street parking areas and any other facilities (e.g., restaurants, cafes, hospitals, etc.). The parking map unit is decomposed into several modules, which are *(i)* map, *(ii)* exporting parking area views into an image file, *(iii)* facility management, *(iv)* parking lot occupancy monitoring, and *(v)* location management. The map and location management modules are further decomposed into sub-modules. The driver unit has to do with the functionalities that any drivers can perform in their car parking management processes. The modules considered here include *(i)* profile management, *(ii)* membership, *(iii)* preference management, *(iv)* notification management, *(v)* billing management, *(vi)* favorite places, *(vii)* parking history, *(viii)* help and support and *(ix)* language. The administration unit has to do with the functionalities that are used by system administrators, system providers, and municipalities for maintaining, monitoring and tracking SPMSs. The administration modules include *(i)* user management (e.g., adding/deleting users), *(ii)* performance monitoring, *(iii)* reporting, *(iv)* payment management (e.g., invoicing, secure payment, etc.), and *(v)* tariff and discount management. The security unit provides services for the rest of the software units for ensuring the security of the data that are exchanged among different software modules. Here, the authentication module is for ensuring that the data are accessed by the correct person under the correct access rights. The integrity module is for ensuring that the system data are always maintained as complete, consistent, and correct. The availability module is for ensuring that the data are always accessible for the authenticated users. Lastly, the data management unit collects data from various sources such as sensors, cameras, and any external systems. The data management unit analyses the collected data and generates useful information for the rest of the software units which require that information for operating their functionalities. The data management unit therefore consists of five different modules for *(i)* collecting data from the environment, *(ii)* storing the collected data, *(iii)* processing those data using different data analytic techniques, *(iv)* making decisions using the processed data and then *(v)* visualizing the decisions.

It should be noted that the relationships among the software units are essentially described in the component and connector viewpoint, which is discussed in the next subsection. As the runnable units of an SPMS product, components each execute a cohesive set of software units and are connected via the dependency connectors.

5.4. Component and Connector Viewpoint

To specify the components that represent the processing units for an SPMS and their interactions (i.e., connectors), we use the layered architecture style [49], which is one of the

most widely used architecture styles for information management systems such as SPMSs and best suited for the organization of SPMS components. With the layered architecture style, components are organized into vertical layers, where each layer has a specific goal (e.g., the presentation layer aiming for visualizing user interfaces to the end users) and consists of a cohesive set of components that aim to achieve the layer's goal. Connectors here are considered as simple communication links between layers and enable the transfer of data and control between the components of any two connected layers. A layer may be connected with the layer right below and the components in the layer above may send data (or transfer control) to the components in the layer below. Note that the components in the layer below cannot make requests from the upper layer components.

To determine the executing components for different layers, we used the module viewpoint definition in Section 5.3, where the software units and their modules that can be used for implementing SPMSs are given. That is, each component can essentially execute one or multiple software units/modules that are relevant to each other. Figure 5 shows the application of the layered style for the SPMS domain. Therein, the presentation layer is specified at the top, which encapsulates the components for executing the visual user interfaces on different platforms (i.e., web and mobile). The middle layer is the business logic of SPMSs, where the components for executing the software units/modules about the main SPMS functionalities that are discussed in Section 5.3 are encapsulated. The business logic components are grouped into six categories that represent the main executing components of SPMSs at run-time. These are *(i)* navigation, *(ii)* reservation, *(iii)* administration, *(iv)* parking map, *(v)* parking lot, and *(vi)* driver. Each component here executes the relevant software units/modules that perform the associated functionalities (e.g., navigation component executing the navigation software unit). Upon receiving a request from the presentation layer components, the business logic layer components handle the request and take any necessary action(s) which may further require any data to be requested from the data management layer components. The bottom layer is the data management layer, whose components execute the data management software units/modules from RA's module viewpoint. The data management layer thus consists of five different components: *(i)* acquiring data from data sources (e.g., sensors and camera), *(ii)* storing the collected data, *(iii)* processing the acquired and stored data using data analytic techniques (e.g., artificial intelligence and statistical techniques), *(iv)* making decisions on the processed data, and lastly, *(v)* visualizing the decisions. Lastly, the security layer that is supposed to execute the security software unit from the module viewpoint is specified. So, the security components here receive requests from all other layers so as to ensure the security of the data managed by an SPMS.

5.5. Allocation Viewpoint

In the allocation viewpoint, the goal is to design how the software components from the component and connector viewpoint can be mapped into the physical components of SPMSs. To this end, we used the client–server architecture style and considered two main types of physical components, which are client and server. A client device can run a component that makes requests for getting some tasks to be done, such as mobile devices for drivers and PC machines for parking providers. The server device can run a component that provides the business logic services. That is, a server can receive requests from a client and makes calculations to respond the client with a result. Note that the server here can be deployed either *(i)* physically on a physical device or *(ii)* virtually in a cloud environment. The server and client devices communicate over the network using the appropriate communication protocols, e.g., HTTP and TCP.

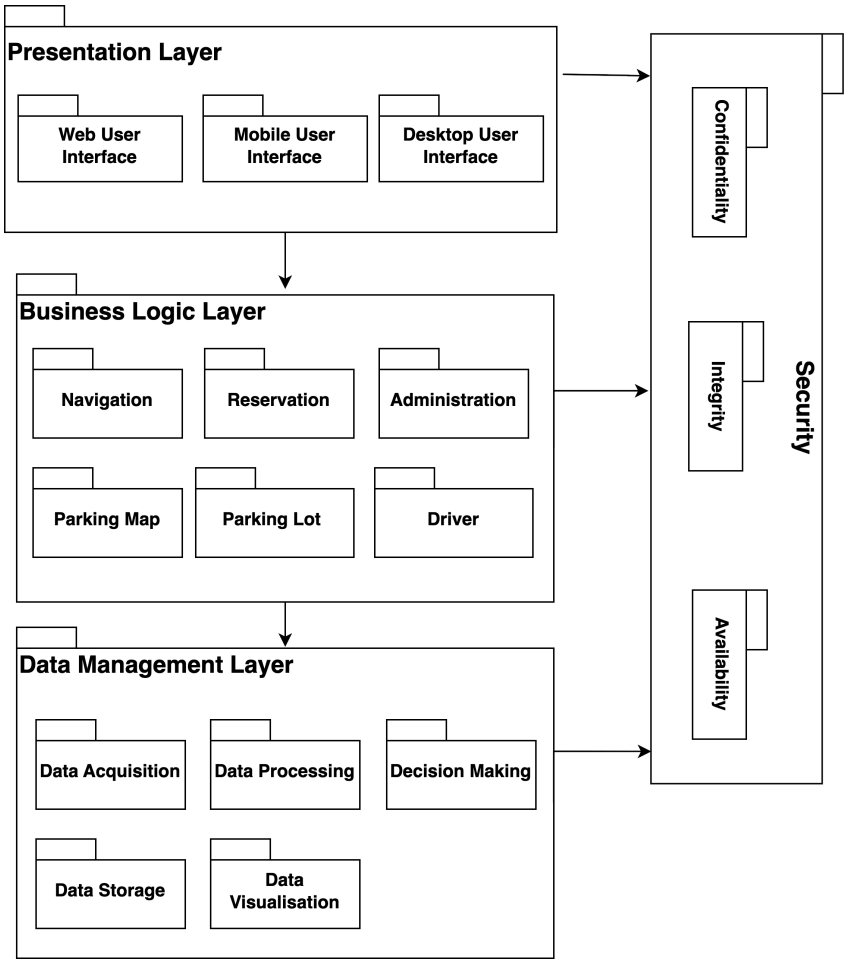

Figure 5. The component and connector viewpoint for SPMS (layered style).

Figure 6 depicts the allocation viewpoint for SPMSs. The presentation layer components (e.g., mobile, web or desktop applications) from the component and connector viewpoint in Figure 5 are mapped into the client devices, while the business logic and data management layer components are mapped into the server devices.

A separate client device can be specified for each type of SPMS user (e.g., driver, parking controller, and parking provider), which may run either of the presentation layer components (i.e., mobile, web, and desktop). Multiple server devices can be specified for the physical architecture of any SPMSs. Also, backup servers or different types of servers (e.g., database server, web server, etc.) can be specified.

In the physical architecture of any SPMSs, intermediate load balancing software applications can also be specified for directing the requests to different servers depending on the current loads of the servers. Furthermore, any data sources (e.g., sensor and camera devices) can be involved in the physical architecture so as to share data with the servers in real time. Lastly, external devices that run any external systems (e.g., electric car charging applications and fuel payment applications) can be involved in the SPMS physical architecture and handle specific tasks that are not supported by the SPMS servers. The data sources and external systems can have their own local database and interact with the SPMS server for communicating their data (or requests).

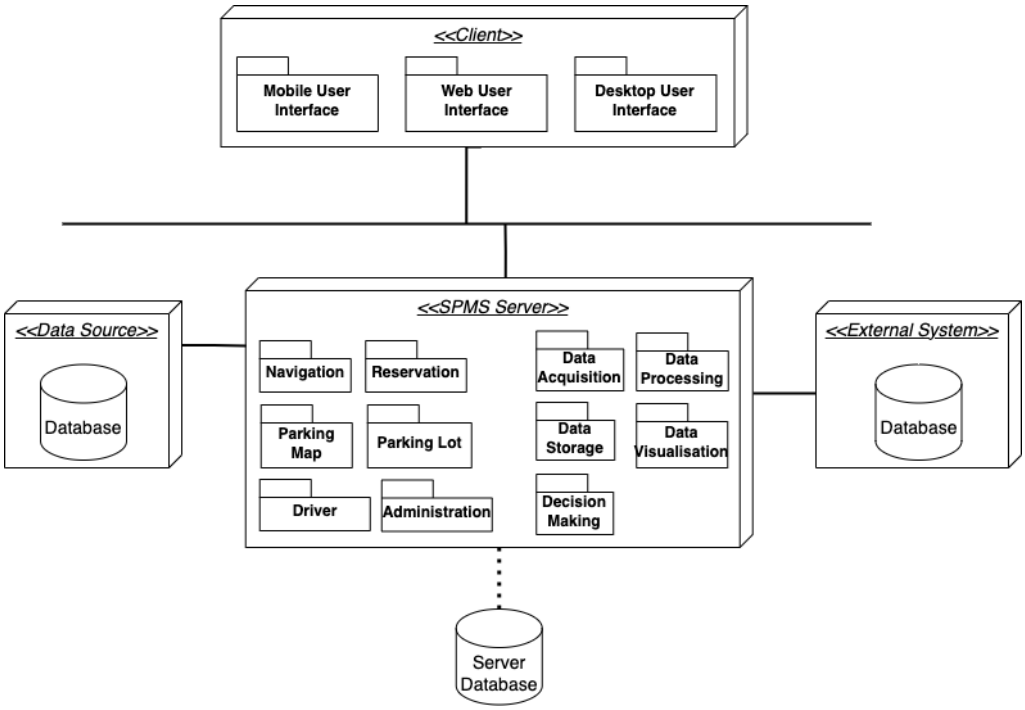

Figure 6. The allocation viewpoint for SPMS.

6. Case Study Evaluations

To evaluate our RA design for the SPMS product family, we considered a real scenario specification and a prototype implementation. In the real scenario specification, we specified an AA of a real SPMS, as discussed in Section 6.1. In the prototype implementation, we developed a prototype SPMS, as discussed in Section 6.2.

To conduct the case studies, we followed the research process proposed by Runeson et al. [50]. Note that Runeson et al. proposed well-regarded steps for performing case studies which have been widely adopted in industry and academia.

6.1. Case Study for the Real Scenario Specification

In this case study, we used the commercial SPMS product called 4Park[6], which is one of the top-used commercial tools for smart car parking and has been developed by the Smart Parking Systems company based in Italy. Our design goal here is to specify the AA for 4Park by re-using our RA design that is introduced in Section 5.

To obtain the domain knowledge and collect data, we went through the 4Park web-site, where case studies, videos, tutorials, white papers, and technical support contacts (e.g., e-mail and phone) are easily accessible. We examined each of those knowledge sources and collected data about the tool features for 4Park.

To analyze the collected data about the 4Park tool features, we specified a feature model by re-using our RA design (Section 6.1.1).

Upon analyzing the feature model for 4Park given in Section 6.1.1, we specified the AA for 4Park by reusing the RA viewpoint definitions. In Sections 6.1.2–6.1.5, we report the view models.

In Section 6.1.6, we measured the reusability ratio for the AA specification of the 4Park SPMS. By doing so, we aimed to demonstrate to what extent our RA design can be reused in specifying the AA of a real SPMS. We strongly believe that specifying the AA of an SPMS by means of reuse can reduce the time and effort needed for analyzing, designing and implementing SPMSs and also increase the level of software quality [51].

6.1.1. Feature Model

Figure 7 depicts the feature model for the 4Park SPMS, which includes all the features that are given in the RA design. Moreover, the parking map feature includes an additional sub-feature for managing the parking locations for disabled people (colored pink)[7].

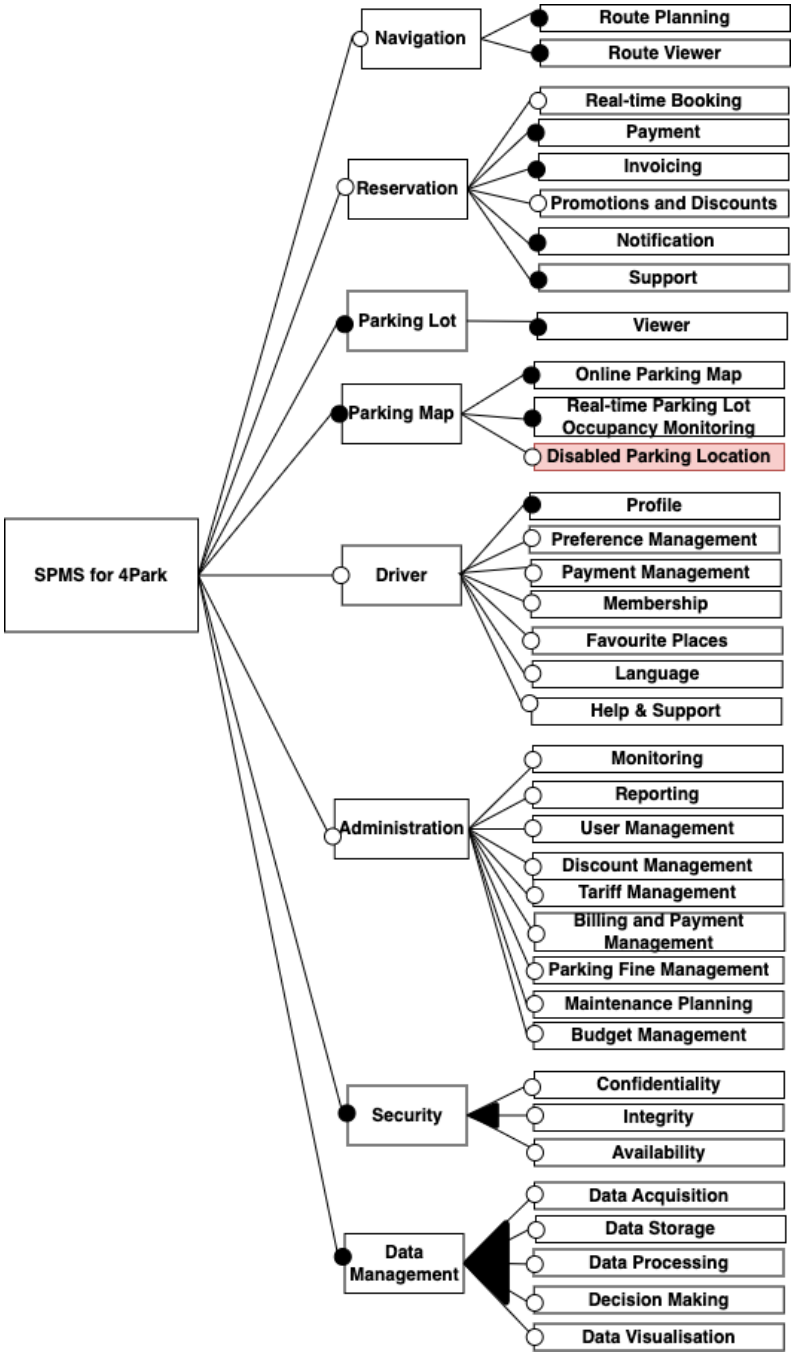

Figure 7. The feature model for 4Park.

While all the main features of the RA feature model that is depicted in Figure 2 have been re-used in 4Park, not all the sub-features of the main features have been re-used. The exceptions here are the administration, security and data management features, all of whose sub-features are re-used by 4Park. However, the navigation feature for 4Park is just considered in terms of route planning and viewer. Also, the parking lot feature does not seem to support designing and editing parking lots. Similarly, the parking map feature

does not support location saving, editing, and sharing, and the management of facilities (e.g., nearest hospitals and cafes).

6.1.2. Context View

The context view for 4Park is modeled in Figure 8. Here, the newly introduced stakeholders that do not exist in RA's context viewpoint are colored in pink, while those stakeholders that already exist in RA's context viewpoint with a different name are colored in blue. So, the SPMS product for 4Park is accessed by the driver, system manager, parking controller, police, administrator, and municipality staff stakeholders. The "system manager" here corresponds to the "system provider" stakeholder that is given in RA's context viewpoint. The sensor and camera devices are used by 4Park's SPMS solution to collect data from car parking areas in real time. The external system that is involved in 4Park is the electric car charging system, which works in an integrated manner with the 4Park SPMS server. 4Park additionally involves the "police" stakeholder, who is notified about any issues. Note that this is enabled by the administration module, which is discussed in the next sub-section.

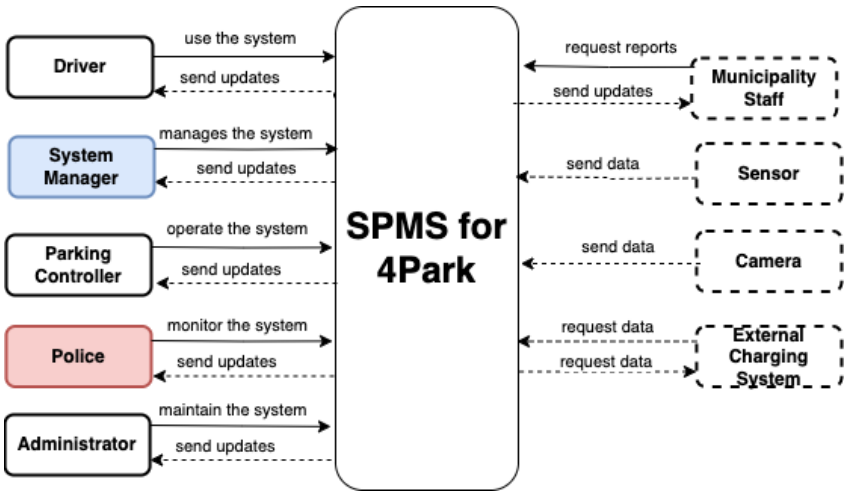

Figure 8. The context view for 4Park.

6.1.3. Module View

The module view for 4Park is modeled in Figure 9. Considering RA's module viewpoint, which is discussed in Section 5.3, 4Park's module view has some differences. Since some of the sub-features defined in RA are not supported by 4Park, as discussed in Section 6.1.1, the associated software modules are not considered in the corresponding software units. For instance, the navigation unit and parking lot unit each include one software module only, despite the RA design considering several modules (and even sub-modules). Note that 4Park includes a different software unit called "Management Software POLIS" (colored pink)[8], which is actually the composition of the existing "Data Management" software unit with the "Administration" software unit. Management Software POLIS manages all the data that are received from various data sources in real time and analyzes those data so as to perform many important operations for system administration. These operations include the real-time monitoring of the car parking spaces, generation of useful reports (e.g., maintenance planning), fine management, payment management, and management of user parking activities. Management Software POLIS provides services for many stakeholders, including parking controllers, system managers, police and administrators. Also, the "parking map" software unit includes a new module called "disabled parking location", whose goal is to enable disabled people to reach convenient parking locations.

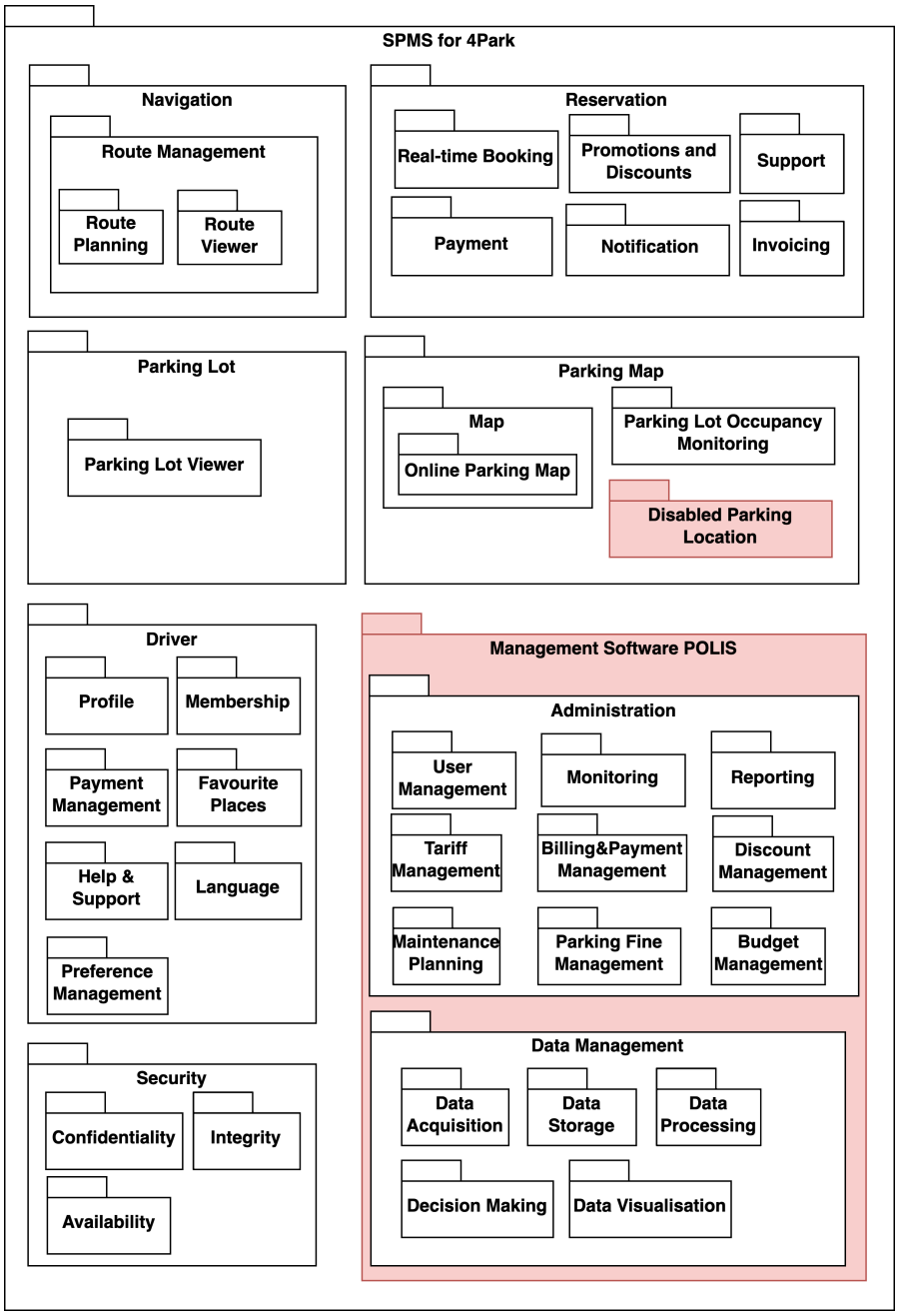

Figure 9. The module view for 4Park.

6.1.4. Component and Connector View

The component and connector view for 4Park is modeled in Figure 10. The view model for 4Park includes an additional layer between the business logic and data management called "Management and Monitoring Layer" (colored pink), compared with RA's component and connector viewpoint definition in Section 5.4. We observed that 4Park considers the management and monitoring as the first-class component of their smart car parking solution, which provides services for the rest of the business logic and user interface components. The management and monitoring layer here encapsulates the administration component, which executes the administration sub-unit of the newly introduced software unit called "Management Software Polis" in Figure 9. The management and monitoring layer therefore offers services that are related to the real-time monitoring of car parking spaces and management facilities (e.g., reporting, fine management, payment management, etc.), which can be requested by the presentation and business logic layer

components. The management and monitoring layer reaches the data maintained by the data management layer and produces knowledge that can be used by the presentation layer and business layer components. Note that the data management layer here encapsulates the components that execute the sub-units which are part of the data management software unit. The data management software unit is essentially part of the "Management Software Polis" software unit in Figure 9 which also includes the administration unit. So this reveals that the execution of the "Management Software Polis" software unit is handled by the components that are encapsulated in two separate layers.

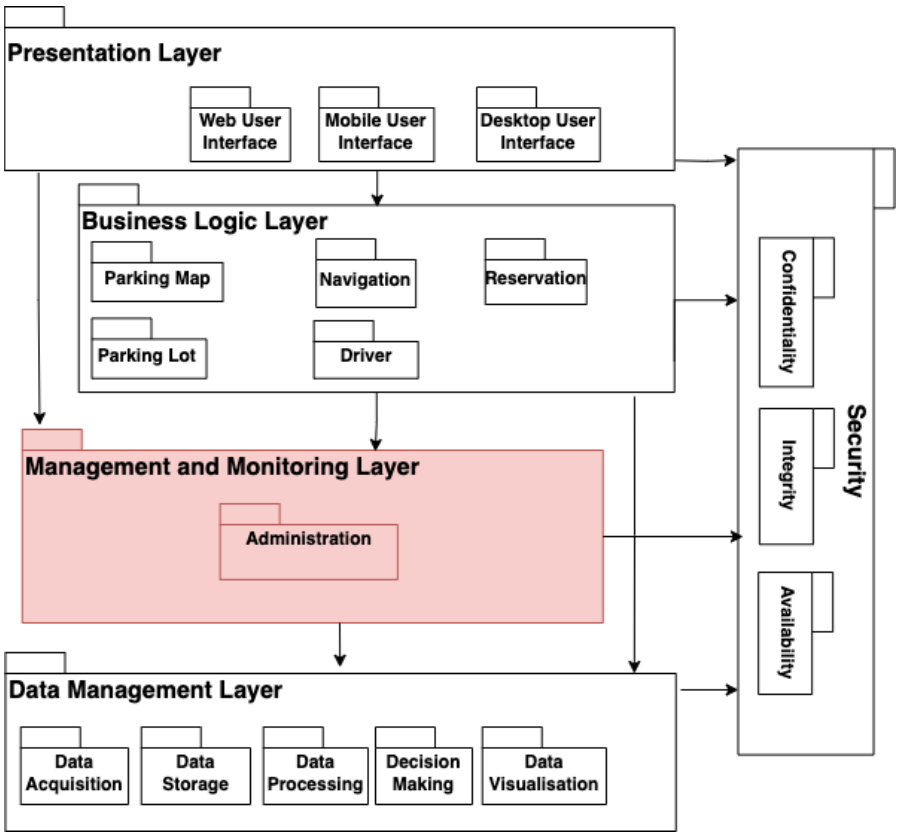

Figure 10. The component and connector view for 4Park.

6.1.5. Allocation View

The allocation view for 4Park is modeled in Figure 11. 4Park's allocation view reuses the client and server elements from the allocation viewpoint definition discussed in Section 5.5. 4Park works with a centralised server that runs on the cloud environment and the server executes all the components of the component and connector view that is given in Figure 10. The server is accessible via different types of clients such as police, municipality staff, driver, car parking manager, and parking controller. While the police, municipality staff, parking controller, and park manager can send requests to the server via mobile, web, and desktop user interfaces, drivers send requests to the server via mobile user interfaces only. Another client is the KIOSK device, which offers a desktop user interface for the drivers to manage their user profiles and perform parking payment.

4Park SPMS uses the sensor and camera data sources to collect the occupancy data from the parking areas in real time and send them to the server. 4Park SPMS is integrated with an electric car charging system, which can provide several services for electric cars and exchange data with the server.

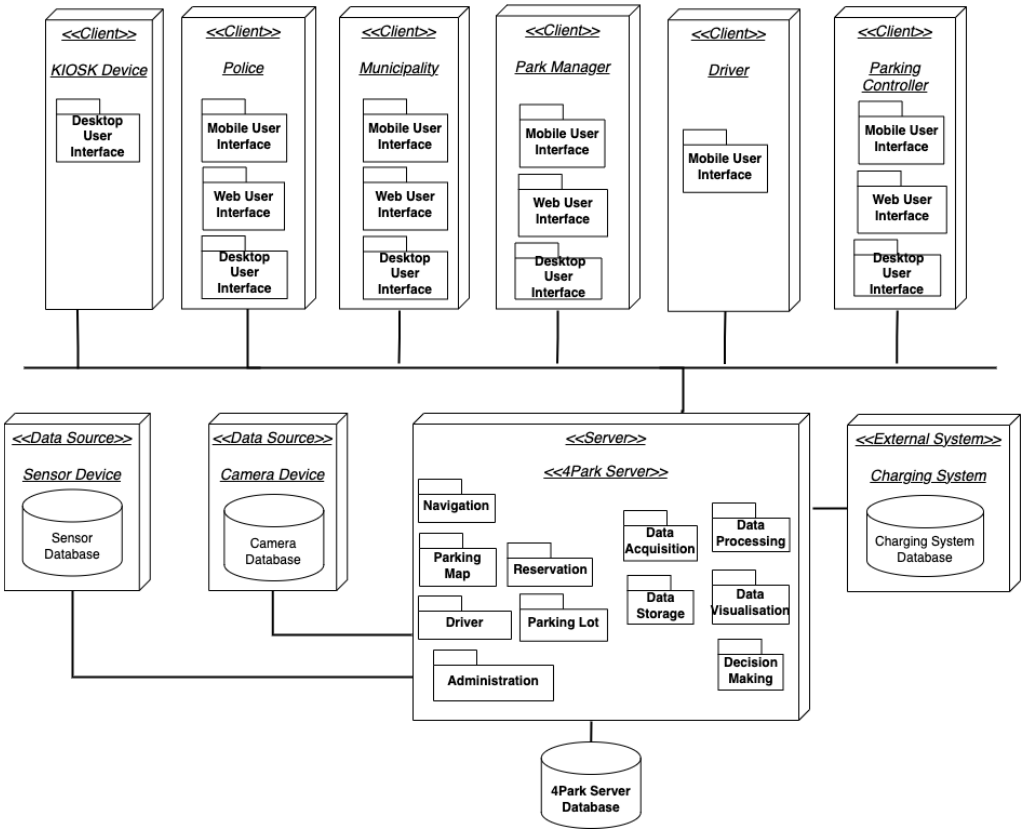

Figure 11. The allocation view for 4Park.

6.1.6. Measuring Reusability

In the previous subsection, we discussed the specifications of the view models for 4Park's AA. To specify each view model, we reused RA's corresponding viewpoint definition to some extent. For instance, while the "parking lot" software unit includes five different modules in the RA design, we reused only one of those modules for 4Park's module view as the functionalities that correspond to the rest of the modules are not supported by 4Park. To calculate the percentage of reuse for each view model, the following formula is used. Therein, we count the number of viewpoint elements that have been used in the AA's view model for each viewpoint and sum the computed values together. Then, we divide that value by the total number of elements that exist in the RA viewpoint definition.

$$\frac{\texttt{\#reused elements in the AA view model}}{\texttt{\#elements in the RA viewpoint definition}} \times 100\%$$

Concerning the context view model, 66% of the stakeholders that are defined in the RA context viewpoint, which is depicted in Figure 3, have been reused by 4Park's context view model in Figure 8. Concerning the module view model, 61% of the software modules that are defined in the RA's module viewpoint, which is depicted in Figure 4, have been reused in the 4Park's module view model in Figure 9. Concerning the component and connector view model, 100% of the components that are defined in the RA's component amd connector viewpoint, which is depicted in Figure 5, have been reused in the 4Park's component and connector view model in Figure 10. Concerning the allocation view model, 100% of the physical components that are defined in the RA's allocation viewpoint, which is depicted in Figure 6 (i.e the client, SPMS server, data source, external system, and server database), have been reused in the 4Park's location view model in Figure 11. So, the percentage of re-use for 4Park is 82% on average.

6.2. Case Study for the Prototype SPMS Development

In this case study, we addressed the parking problem that we observed in Bagdat Avenue, Istanbul[9], which negatively affects thousands of people's lives. We developed a prototype SPMS that can be used by the parking controllers and drivers for the effective use of the parking spaces around tens of different streets in Bagdat Avenue. Using the SPMS prototype, parking controllers can mark the available parking spaces on their streets. Drivers can detect available spaces, perform reservations that are to be confirmed by the parking controllers, and navigate to the reserved parking spaces.

To collect the domain data, we conducted several informal interviews with the parking controllers and drivers in Bagdat Avenue. Also, we used the survey results that we have obtained previously[10]. We analyzed the collected data from our interviews and surveys and determined the requirements for the SPMS prototype.

In Section 6.2.1, the processes of developing the SPMS prototype are reported in terms of two subsequent stages. In the first stage, we asked one team to analyze the requirements and develop an SPMS prototype without using our RA design. In the next stage, we asked another team to develop an SPMS prototype by using the RA design this time. In Section 6.2.2, we measured the development performances of the two separate groups in terms of the time spent on different phases of the SPMS development life cycle (i.e., analysis, design, implementation and testing).

6.2.1. Developing an SPMS Prototype

To develop an SPMS prototype that meets the desired requirements, the software engineering research group (SERG) at Yeditepe University[11] has volunteered. Firstly, we asked five senior undergraduate students who study computer science and are members of SERG to design and implement an SPMS prototype without using the RA design. It should be noted that we selected the students who scored best in the programming courses and work as part-time developers in industry. We conducted a meeting with the students to discuss the requirements. To communicate the requirements, we used a whiteboard where we sketched some diagrams and wrote some supplementary texts. The meeting was interactive, and thus, the students asked questions about any ambiguities. The meeting took approximately two hours to complete. At the end of the meeting, the students agreed that they could develop an SPMS prototype in four months.

Unfortunately, the students were not able to deliver an SPMS prototype and announced the failure of their project after 80 days, with no concrete output. The post-interviews with the students after the project failure shed light on several problems that the students faced during the software development processes. The students seemed to have underestimated the SPMS domain and failed to clearly understand the needs of different stakeholders. So, this made them use their time unproductively and caused them to lose motivation throughout the project. Another issue has to do with the lack of modeling and design efforts spent during the project. Indeed, since the students started coding immediately, they omitted several requirements and principal design decisions. Also, the students' decomposition of the entire SPMS problem into the sub-problems was not so effective either, which caused unnecessary dependencies and prevented the students from working together effectively and integrating their solutions. The students further pointed out that they could not manage to prioritize the problems at the beginning, and thus, they underestimated the efforts required for the most difficult tasks such as parking lot design and occupancy monitoring. As a result, the students ended up with a bunch of code that represents the implementation of some system requirements which cannot successfully be integrated into a whole system. The code has been written in a sloppy manner, which is not easy to understand and analyze either.

Concerning the second half of the evaluation, we selected a completely different group of five students. Likewise, the new students are top-ranked in the programming courses and work as part-time developers in different industries. We initially provided the students with our RA design document this time. We conducted a meeting with the students and introduced them to our RA design. Furthermore, we explained the requirements of the desired SPMS solution. The students agreed on using the RA design and developing the desired SPMS solution accordingly. Our observations showed that the students were happy with the RA design and willing to use RA for different stages of their SPMS development processes (e.g., requirement analysis and early architectural design). Our RA helped students in performing crucial activities such as the decomposition of problems into sub-problems, prioritizing the problems, and tracking and monitoring the work completed by the team members. By doing so, the students succeeded in delivering an SPMS prototype in just three months (approximately 90 days). Figure 12 shows some of the user interface visualizations from the delivered SPMS prototype[12].

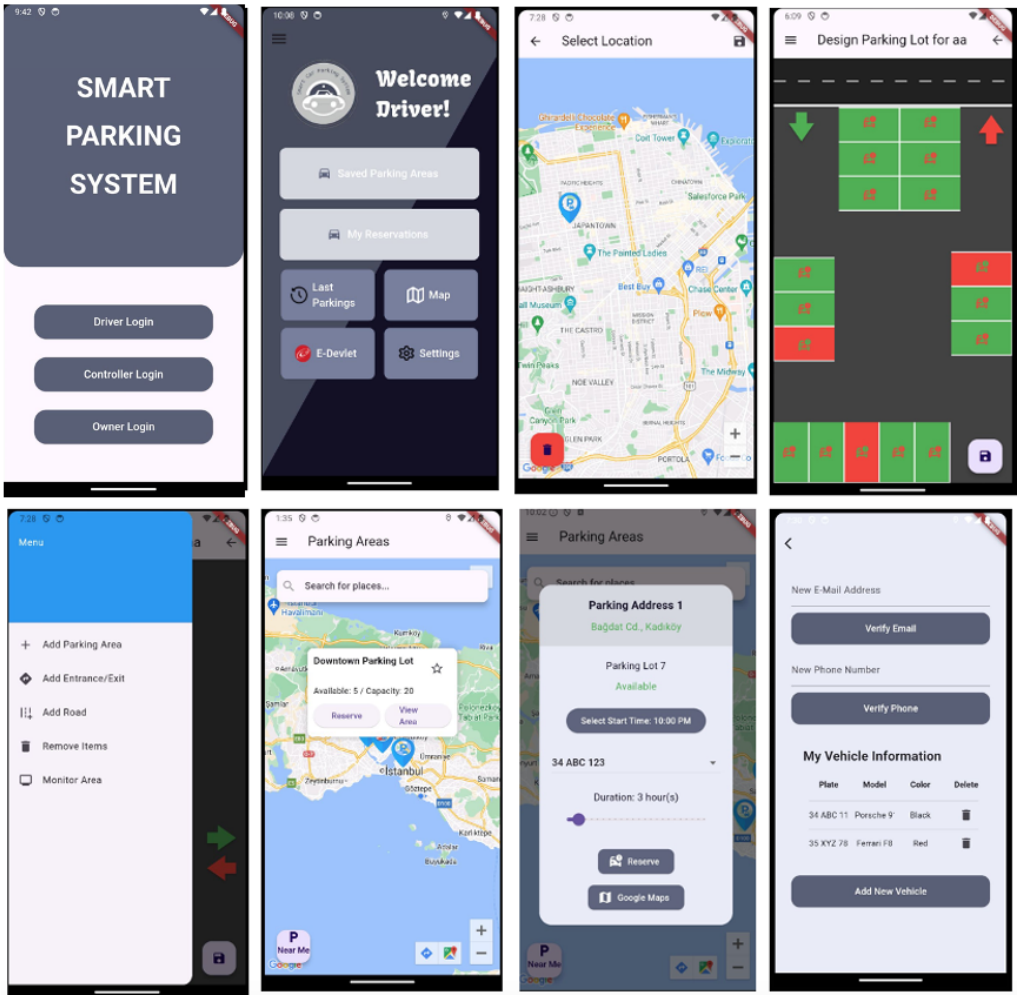

Figure 12. The user interface pictures of the SPMS prototype.

6.2.2. Measuring the Development Performance

Figure 13 shows the number (and percentage) of days the students spent on different stages of the SPMS prototype development (i.e., analysis, design, implementation and testing). Note that we considered here both the case when the prototype was developed with RA and the other case when RA was not used. So, using RA led to the successful development of the SPMS prototype in 90 days, while the case when RA was not used resulted in no running prototype after 80 days of effort. In the case when the RA design was

not used, the students indicated that they spent just 10% of their total efforts on the analysis of SPMS requirements, and the rest of their efforts were spent on the SPMS implementation. This is because the students had no any guideline for the architectural design of SPMS solutions and attempted the implementation immediately. When our RA design was used, the amount of effort spent for the requirement analysis and early architectural design grew higher (i.e., 25% of their total efforts). Indeed, the students found the RA viewpoint definitions highly useful in understanding the stakeholders and their roles, prioritizing the tasks, and making design decisions on the system structures and their interactions before coding. By doing so, the students' efforts in the implementation were reduced to 36 days and a fully executable SPMS product was delivered in the end. According to the students, early structuring of the software units is the main factor that reduced their implementation efforts, as the code dependencies are minimized.

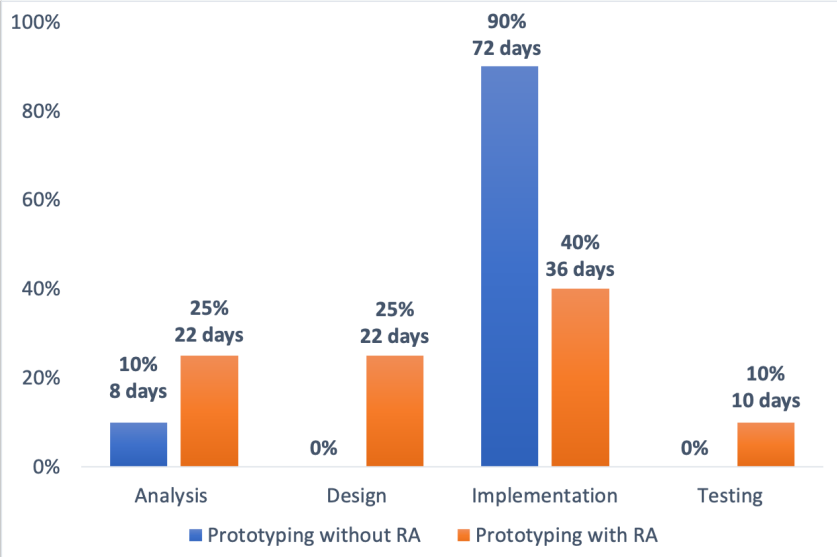

Figure 13. Comparing the development performance with RA and without RA.

7. Discussion

7.1. Summary of Findings

To the best of our knowledge, our RA design for the SPMS product family that is introduced in this paper is novel as being the only generic approach that can be re-used as a guideline for designing the architectures of SPMSs. As discussed in Section 2, the existing approaches in the literature do not aid in understanding *(i)* which types of stakeholders are involved in the SPMS domain and their interactions with the system, *(ii)* the common and varying features for SPMSs, *(iii)* the software units that can be designed for implementing an SPMS, *(iv)* the architecture styles that can be used for designing the software architecture of an SPMS, and *(v)* the styles for mapping the software units into the physical devices. Therefore, it is strongly believed that this paper will bridge the gap here and facilitate the architecture design of any SPMS solutions.

In the rest of this section, we discuss how we satisfy the research questions (RQs) which are given in Section 3 and any lessons that we learned.

To address RQ1 in Section 3, we performed a comprehensive analysis of the SPMS domain, which includes the analysis of the literature and commercial SPMS tools. So, we specified a large set of SPMS features, which can be categorized as mandatory features and optional features. To validate the feature set, we conducted a survey that is supported with face-to-face interviews with 15 parking controllers who work for ISPARK in Istanbul, Türkiye, which is the main parking provider controlled by the municipality. With the

survey results, we were able to revise our feature set for the SPMS product family and detected any inconsistencies and redundancies. We further decomposed each feature into a set of sub-features that aid in achieving the goal of that feature (e.g., route optimization for the navigation feature). Note, however, that to avoid any complexities, no sub-sub-features have been considered, as this could make the RA design difficult to manage and communicate. Moreover, our domain analysis and the resulting feature model for SPMSs further helped in addressing RQ2. That is, we determined the stakeholders who can be involved in the development, operation, and maintenance phases of an SPMS product life cycle.

Upon specifying the feature model for SPMSs, we defined the viewpoints of our RA so as to address RQ3. We have been inspired by Clements et al.'s seminal book on architecture viewpoints [34] and thus decomposed our RA design into five distinct viewpoints, each of which describes the architecture of an SPMS from a particular point of view. These are *(i)* context, *(ii)* module, *(iii)* component and connector, and *(iv)* allocation. It should be noted that Clements et al.'s viewpoints are considered as a standard technique for the effective specifications of RAs [47,52,53]. Indeed, many RA studies in the literature that focus on diverse problem domains use the same set of viewpoints, e.g., RA for smart farming [29], RA for digital twins [54], RA for ontology engineering tools [48], and RA for smart warehouses [46].

In the context viewpoint of our RA design, we essentially aimed to address RQ2 and describe all the stakeholders for SPMSs, their roles and relationships with an SPMS. That is, an SPMS stakeholder can be either optional or mandatory; each stakeholder can be in one of the following forms: *(i)* human (e.g., driver and parking controller), *(ii)* external system (e.g., fuel payment and car charging applications), or *(iii)* data source (e.g., sensors and camera). Stakeholders' relationship with an SPMS is described in terms of the purpose of the relationship, the relationship status (i.e., either mandatory or optional), and the data flow direction (i.e., either from SPMS to stakeholder or vice versa).

In the RA design's component and connector viewpoint, the layered architecture style has been adopted for the organizations of components and connectors. That is, the components that represent the executing units of SPMSs are grouped as layers and each component in one layer may only send requests to the component(s) of the layer(s) below. The layered architecture style is one of the top-used architecture patterns in industry and facilitates the understandability of the system components and their interactions. Moreover, the structural organizations of information systems (e.g., SPMSs) fit very well with the layered style, where the top layer represents different user interfaces which require services from the main logic layer (e.g., a driver requesting to perform a reservation), and that in turn requires services from the data layer (e.g., the reservation component requesting data about the park occupancies).

To validate our RA design in accordance with RQ4, we considered two crucial development quality properties, which are reusability and development performance. In Section 6.1, we focused on evaluating how our RA design can be re-used systematically so as to reduce the effort in specifying the AAs of any SPMS products. To this end, we used a real scenario that is based on a commercially available SPMS called 4Park. Firstly, we reused the feature model of the RA design and determined the features and sub-features that are supported by 4Park. Then, we specified 4Park's AA by reusing the RA viewpoint definitions. With the context viewpoint, we determined the stakeholders who are involved in the context of the 4Park business domain. After specifying the context view model for 4Park, we specified the module, component and connector, and deployment views for 4Park by reusing the corresponding viewpoint definitions. Note that we removed any elements that are not needed for 4Park but have already been defined in the RA viewpoint definitions.

In total, we managed to reuse 82% of the RA design elements in the AA specifications for 4Park (Section 6.1.6). While reusing RA for the AA specifications of 4Park, we performed the following operations: *(i)* composing/decomposing the existing elements from the RA viewpoints (e.g., software units in the module viewpoint) or *(ii)* adding new elements (e.g., adding a new intermediate layer for the component and connector viewpoint).

In the second phase of our validation discussed in Section 6.2, we focused on evaluating how our RA design aids in improving the development performance for the SPMS products. To this end, we performed a prototype evaluation where one group developed an SPMS without RA and another group developed an SPMS with RA. Our evaluation results show that using RA encourages developers to spend their efforts on the early stages of the SPMS development (i.e., the requirement analysis and design stages) and thus reduces the efforts needed for the product implementation. It should also be noted that using RA improved the chance of having a successful product at the end, as any unforeseen errors can be prevented with the early analysis and design. With the use of our RA design, customers and software development teams can communicate effectively on a specific problem and perform the required engineering activities precisely, including gathering, analyzing, and prioritizing the requirements. Furthermore, our RA design facilitates the early architectural design decisions regarding the system structure and interactions, which can considerably reduce the efforts spent on the software implementation. This is because software development teams can *(i)* determine the problems to be solved, their levels of criticality, and the dependencies between different problems, *(ii)* make crucial design decisions for maximizing the reusability of software implementation and *(iii)* easily integrate different parts of the implementation.

The process of designing an RA for SPMSs can essentially be considered as a domain-driven design (DDD) [55], since our RA design required us to perform a comprehensive analysis of the SPMS domain first and define a set of abstract modeling viewpoints with the use of the domain features. With our RA design for SPMSs, we do not consider any low-level details such as the choice of technologies to be used for implementing SPMSs (e.g., programming, database, and networking technologies) and low-level design decisions (e.g., the optimal data structures and algorithms). The main focus with our RA design is on proposing a generic, architecture-level solution using abstract modeling. Therefore, our RA design for SPMSs is essentially useful in the initial stages of SPMS development life-cycles. Indeed, with our RA design, the communications between non-technical stakeholders (e.g., users, customers, and managers) and technical stakeholders (e.g., developers) are facilitated, the system boundaries can easily be established, and any high-level design decisions can easily be figured out. It should be noted that without making early architectural decisions in a systematic way, low-level design decisions (e.g., algorithms and data structures) cannot necessarily be optimized, which therefore leads to project failures.

We strongly believe that our RA design for SPMSs will be very useful for many stakeholders. Any investors and high-level managers of SPMS development companies can use our RA design and agree on the features of any SPMS product to be built and its boundaries, which aid in the early estimation of the time and budget needed. Software developers can use our RA design and figure out *(i)* the software units that are needed for developing an SPMS with the expected requirements and *(ii)* any principal design decisions about the logical and physical architectures and their traceabilities with the software units. By doing so, different phases of SPMS development (i.e., analysis, design and implementation) can be performed systematically and high-quality SPMS products can be developed, as demonstrated in Section 6. Moreover, tool vendors can use our RA design and understand the missing features in their SPMS tools. Also, researchers can gain inspiration from our RA design and conduct relevant research, such as *(i)* proposing RAs

for other problem domains, *(ii)* conducting surveys on the existing SPMS technologies and *(iii)* analyzing the SPMS technologies with regard to the feature set that is proposed in the paper. Lastly, university instructors can use our RA as part of their software architecture courses and demonstrate the use of software architectures on an interesting problem.

7.2. Threats to Validity

7.2.1. Construct Validity

Construct validity is concerned with the extent to which our RA design for SPMSs is successful in reaching our research goal. The main goal for designing an RA for SPMSs is essentially to propose generic guidance that can be re-used for specifying the AAs of SPMS solutions in a systematic way and aids in improving the SPMS development processes.

To minimize any threats for the construct validity, we investigated four essential research questions (RQs) for achieving our research goal here. In Section 3, each research question is discussed in terms of its motivation and how it is aimed to be achieved. Also, to design our RA for SPMSs in the right way, we followed the domain-driven design technique, which is followed by many other RA proposals in the literature (e.g., [28,29,46–48]). Moreover, to manage the complexity of our RA design, we adopted a multiple-viewpoints approach that is based on Clements et al.'s seminal book on architecture viewpoints [34]. It should also be noted that Clements et al.'s approach can be considered as a de facto standard for RA designs, and our motivation for the selection of viewpoints is discussed thoroughly in Section 5.1.

Our RA's module viewpoint, which is discussed in Section 5.3, proposes eight different software units which encapsulate a cohesive set of software modules. While the dependency relationships between software modules are also important for understanding how those modules are composed together, we do not consider any relationships in our module viewpoint definitions. We believe that this could make the viewpoint description complex, considering several modules that exist in our RA's module viewpoint. Moreover, many other RA studies that focus on the generic design of complex smart systems abstract the module relationships from their module viewpoint definitions, e.g., [28,29]. It should also be noted that the relationships between any two software units of SPMSs are essentially addressed by RA's component and connector viewpoint, where components encapsulate the software units that they can execute and connectors represent the dependency relationships between the components.

7.2.2. Internal Validity

Internal validity is concerned with the existence of any unknown variables that may not be considered in our RA design for SPMSs but cause the RA design to be biased.

To minimize any threats here, we performed a comprehensive domain analysis in a systematic way, as discussed in Section 4. That is, we analyzed the literature by considering many papers that discuss parking management techniques and technologies. Also, we further analyzed many commercial SPMS solutions in terms of their features. Then, we conducted a local survey among parking controllers so as to understand their expectations and challenges in terms of parking management. The survey here aids us in validating our proposed feature set for SPMSs. While we believe that our methodology for identifying and analyzing the SPMS features was successful, mathematical approaches could be considered in the future for evaluating the quality of SPMS features and maximizing their precision, such as Principle Component Analysis [56] and Multi-criteria Decision Making [57]. By doing so, the feature set for SPMS can be determined in the most precise and accurate way possible through the applications of systematic processes that are formally defined.

To minimize any threats against RA structuring, we examined various RA samples in different domains (e.g., [28,29,46–48]) and further analyzed their research methodologies and the RA structuring techniques. Indeed, several RA designs in the literature use Clements et al.'s viewpoint definitions [34], as we do in our RA.

In Section 6, we discussed the case study evaluations in terms of a real scenario and prototype software development. In the real scenario evaluation, we measured our RA's reusability by counting the number of RA elements that are reused in the AA specification of the real scenario. While we believe that our method for measuring reusability is commonly applied for RA evaluations (e.g., [29]), there could be more precise ways for evaluating the reusability that can be applied in our future studies. Indeed, to enhance the precision of the reusability measurement, one could consider a weight value for the reused elements that can be determined based on, e.g., the elements' status being common or optional and the number of stakeholders that are associated with the elements. In the prototype evaluation, we measured the development performance by comparing two independent groups, where one group has been asked to develop an SPMS without our RA design and another group has been asked to develop an SPMS by using the RA design. Here, we selected the group members from senior undergraduate students who also work as part-time software developers. While this could cause some biases, we did not have the chance to find more experienced engineers who work full-time and are willing to spend effort developing an SPMS tool for our empirical study. Also, to reduce any biases, we ensured that the two groups consisted of different students with a similar level of programming experience.

7.2.3. External Validity

External validity is concerned with the applicability of our RA design to different SPMS problems.

In Section 6, we demonstrated the applicability of our RA design via two different case studies. Firstly, we used a commercial SPMS application called 4Park and specified the AA for 4Park using our RA design. Then, we analyzed, designed and implemented an SPMS prototype for a real problem by using our RA design as guideline throughout the entire development life cycle.

8. Conclusions

In this paper, we proposed an RA for the SPMS product family, which represents architectural guidance that can be utilized for the analysis, design, and implementation of SPMS solutions in a systematic way. Upon performing a comprehensive domain analysis, we proposed a feature model for the SPMS product family, which describes the common and varying features and their sub-features for SPMSs. Using the feature model, we defined the four architecture viewpoints for our RA design, which are *(i)* context, *(ii)* module, *(iii)* component and connector, and *(iv)* allocation. The context viewpoint is concerned with specifying the system boundaries in terms of the stakeholders and their interactions with an SPMS. The module viewpoint is concerned with specifying the software units that can be designed and implemented for an SPMS and their software modules. The component and connector viewpoint is concerned with specifying the run-time structure of an SPMS in terms of the components that represent different processing units for an SPMS and the connectors for the component interactions. The allocation viewpoint is concerned with specifying the physical architecture and the mapping between the components and physical devices.

We evaluated our RA design for understanding to what extent our RA improves the design reusability and development performance. Firstly, we considered a commercial

SPMS called 4Park and specified its AA by means of reusing the RA viewpoints. So, this aids in understanding to what extent our RA can be reused for specifying the AA of an SPMS solution. Then, we developed an SPMS prototype that focuses on a real problem under the guidance of our RA design. The resulting prototype aids in understanding how our RA improves the development performance of SPMS developers during the analysis, design and implementation stages.

Our RA design for SPMSs is believed to be a unique contribution to the literature and can aid developers in analyzing and designing high-quality SPMS products with minimum costs. Our RA can be useful for SPMS tool vendors who can identify the missing features of their tools and evolve their tools from an architectural perspective. Researchers can use our RA to initiate new research projects on car parking management. Lastly, university instructors can use our RA as part of their software architecture courses so as to demonstrate the use of software architectures in a real and interesting context.

In the future, our first goal is to apply our RA design for the development of an SPMS solution that is going to be used in a particular city in Turkiye which suffers from car parking issues. Furthermore, we are planning to evaluate our RA design using diverse SPMS tools that have been developed for different problems and determine any missing features and viewpoints. Another future plan is to design an RA for SPMSs using different architecture models (e.g., C4 [58] and Arcadia [59]). We can then compare the RA designs with our current RA design for SPMSs in terms of the important quality concerns such as understandability, precision, complexity management, reusability, and maintainability. We are also planning to develop a domain-specific modeling language for modeling the architecture of SPMSs in accordance with our RA design. The modeling language here will also be supported with a modeling editor and code generators. Lastly, we will define design patterns for the SPMS domain through which the SPMS software units can be implemented in the most reusable and maintainable way.

Author Contributions: Conceptualization, M.O.; methodology, M.O; validation, M.O.; investigation, M.O.; resources, M.O. and A.T.; data curation, M.O. and A.T.; writing—original draft preparation, M.O.; writing—review and editing, M.O. and A.T.; visualization, M.O. and A.T.; supervision, M.O.; project administration, M.O.; All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: As Alper Turunc is employee of the company DFDS. The remaining author declares that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Notes

1 Fleximodo: https://www.fleximodo.com/.
2 Parkable: https://parkable.com/.
3 4Park: https://smartparkingsystems.com/.
4 ISPARK: https://ispark.istanbul/.
5 The survey questions are accessible here: https://zenodo.org/records/12704679.
6 4Park Website: https://smartparkingsystems.com/en/4-park-smart-parking-app/.
7 The sub-feature for the disabled parking location management: https://smartparkingsystems.com/en/disabled-parking-management/.
8 4Park's management software polis: https://smartparkingsystems.com/en/management-software-polis/.
9 Bagdat Avenue, Istanbul: https://en.wikipedia.org/wiki/Bağdat_Avenue.
10 see Note 5.
11 SERG at Yeditepe: serg.yeditepe.edu.tr.

[12] Demonstration video for the SPMS prototype can be accessed using the link: https://www.youtube.com/watch?v=rOWZSe43Mc0.

References

1. Population, C. Major Agglomerations of the World. 2024. Available online: https://www.citypopulation.de/en/world/agglomerations/ (accessed on 30 May 2024).
2. Isa, N.; Yusoff, M.; Mohamed, A. A Review on Recent Traffic Congestion Relief Approaches. In Proceedings of the 2014 4th International Conference on Artificial Intelligence with Applications in Engineering and Technology, Kota Kinabalu, Malaysia, 3–5 December 2014; pp. 121–126. [CrossRef]
3. Forum, W.E. The Number of Cars Worldwide is Set to Double by 2040. 2016. Available online: https://www.weforum.org/agenda/2016/04/the-number-of-cars-worldwide-is-set-to-double-by-2040 (accessed on 30 May 2024).
4. Lee, J.B.; Agdas, D.; Baker, D. Cruising for parking: New empirical evidence and influential factors on cruising time. *J. Transp. Land Use* **2017**, *10*, 931–943. [CrossRef]
5. INRIX. Searching for Parking Costs the UK £23.3 Billion a Year. 2017. Available online: https://inrix.com/press-releases/parking-pain-uk (accessed on 30 May 2024).
6. Das, S. A Novel Parking Management System, for Smart Cities, to save Fuel, Time, and Money. In Proceedings of the 2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 7–9 January 2019; pp. 950–954. [CrossRef]
7. Shoup, D.C. Cruising for parking. *Transp. Policy* **2006**, *13*, 479–486. [CrossRef]
8. Lin, T.; Rivano, H.; Le Mouël, F. A Survey of Smart Parking Solutions. *IEEE Trans. Intell. Transp. Syst.* **2017**, *18*, 3229–3253. [CrossRef]
9. Ladyman, J.; Wiesner, K. *What Is a Complex System?* Yale University Press: New Haven, CT, USA, 2020.
10. Garlan, D.; Shaw, M. *An Introduction to Software Architecture*; Technical Report; Carnegie Mellon University: Pittsburgh, PA, USA, 1994.
11. Muller, G. *A Reference Architecture Primer*; Technical Report; University of South-Eastern Norway-NISE: Kongsberg, Norway, 2020. Available online: https://www.gaudisite.nl/ReferenceArchitecturePrimerPaper.pdf (accessed on 10 September 2024).
12. Kotb, A.O.; Shen, Y.C.; Zhu, X.; Huang, Y. iParker—A New Smart Car-Parking System Based on Dynamic Resource Allocation and Pricing. *IEEE Trans. Intell. Transp. Syst.* **2016**, *17*, 2637–2647. [CrossRef]
13. Mohammadi, F.; Nazri, G.A.; Saif, M. A Real-Time Cloud-Based Intelligent Car Parking System for Smart Cities. In Proceedings of the 2019 IEEE 2nd International Conference on Information Communication and Signal Processing (ICICSP), Weihai, China, 28–30 September 2019; pp. 235–240. [CrossRef]
14. Kianpisheh, A.; Mustaffa, N.; Limtrairut, P.; Keikhosrokiani, P. Smart parking system (SPS) architecture using ultrasonic detector. *Int. J. Softw. Eng. Appl.* **2012**, *6*, 55–58.
15. Thomas, T.; Bhatt, T. Smart Car Parking System Using Convolutional Neural Network. In Proceedings of the 2018 International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, 11–12 July 2018; pp. 172–174. [CrossRef]
16. Thomas, D.; Kovoor, B.C. A Genetic Algorithm Approach to Autonomous Smart Vehicle Parking system. *Procedia Comput. Sci.* **2018**, *125*, 68–76. [CrossRef]
17. Tandon, R.; Gupta, P.K. Optimizing Smart Parking System by Using Fog Computing. In *Advances in Computing and Data Sciences, Proceedings of the Third International Conference, ICACDS 2019, Ghaziabad, India, 12–13 April 2019*; Singh, M., Gupta, P., Tyagi, V., Flusser, J., Ören, T., Kashyap, R., Eds.; Springer: Singapore, 2019; pp. 724–737.
18. Thakre, M.P.; Borse, P.S.; Matale, N.P.; Sharma, P. IOT Based Smart Vehicle Parking System Using RFID. In Proceedings of the 2021 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 27–29 January 2021; pp. 1–5. [CrossRef]
19. Fahim, A.; Hasan, M.; Chowdhury, M.A. Smart parking systems: Comprehensive review based on various aspects. *Heliyon* **2021**, *7*, e07050. [CrossRef] [PubMed]
20. Paidi, V.; Fleyeh, H.; Håkansson, J.; Nyberg, R.G. Smart parking sensors, technologies and applications for open parking lots: A review. *IET Intell. Transp. Syst.* **2018**, *12*, 735–741. [CrossRef]
21. Xiao, X.; Peng, Z.; Lin, Y.; Jin, Z.; Shao, W.; Chen, R.; Cheng, N.; Mao, G. Parking Prediction in Smart Cities: A Survey. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 10302–10326. [CrossRef]
22. Barriga, J.J.; Sulca, J.; León, J.L.; Ulloa, A.; Portero, D.; Andrade, R.; Yoo, S.G. Smart Parking: A Literature Review from the Technological Perspective. *Appl. Sci.* **2019**, *9*, 4569. [CrossRef]
23. Fraifer, M.; Fernström, M. Designing an IoT smart parking prototype system. In Proceedings of the Thirty Seventh International Conference on Information Systems, ISCA, Dublin, Ireland, 11–14 December 2016; pp. 2–12.

24. Anderson, E.C.; Okafor, K.C.; Nkwachukwu, O.; Dike, D.O. Real time car parking system: A novel taxonomy for integrated vehicular computing. In Proceedings of the 2017 International Conference on Computing Networking and Informatics (ICCNI), Lagos, Nigeria, 29–31 October 2017; pp. 1–9. [CrossRef]
25. Soegoto, E.S.; Pamungkas, V.Y.; Herdiawan, A. Designing Smart Parking Application for Car Parking Space Arrangement. *IOP Conf. Ser. Mater. Sci. Eng.* **2018**, *407*, 012185. [CrossRef]
26. Awaisi, K.S.; Abbas, A.; Zareei, M.; Khattak, H.A.; Khan, M.U.S.; Ali, M.; Din, I.U.; Shah, S. Towards a Fog Enabled Efficient Car Parking Architecture. *IEEE Access* **2019**, *7*, 159100–159111. [CrossRef]
27. Alharbi, A.; Halikias, G.; Yamin, M.; Abi Sen, A. Web-based framework for smart parking system. *Int. J. Inf. Technol.* **2021**, *13*, 1495–1502. [CrossRef]
28. Tummers, J.; Tobi, H.; Catal, C.; Tekinerdogan, B. Designing a reference architecture for health information systems. *BMC Med. Inform. Decis. Mak.* **2021**, *21*, 210. [CrossRef] [PubMed]
29. Krisnawijaya, N.N.K.; Tekinerdogan, B.; Catal, C.; van der Tol, R. Reference architecture design for developing data management systems in smart farming. *Ecol. Inform.* **2024**, *81*, 102613. [CrossRef]
30. Behere, S.; Törngren, M. A functional reference architecture for autonomous driving. *Inf. Softw. Technol.* **2016**, *73*, 136–150. [CrossRef]
31. Tao, F.; Sun, X.; Cheng, J.; Zhu, Y.; Liu, W.; Wang, Y.; Xu, H.; Hu, T.; Liu, X.; Liu, T.; et al. makeTwin: A reference architecture for digital twin software platform. *Chin. J. Aeronaut.* **2024**, *37*, 1–18. [CrossRef]
32. McManus, J. A stakeholder perspective within software engineering projects. In Proceedings of the 2004 IEEE International Engineering Management Conference (IEEE Cat. No.04CH37574), Singapore, 18–21 October 2004; Volume 2, pp. 880–884. [CrossRef]
33. Rozanski, N.; Woods, E. *Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives*, 2nd ed.; Addison-Wesley Professional: Boston, MA, USA, 2011.
34. Clements, P.; Garlan, D.; Bass, L.; Stafford, J.; Nord, R.; Ivers, J.; Little, R. *Documenting Software Architectures: Views and Beyond*; Pearson Education: Boston, MA, USA, 2002.
35. Avgeriou, P. Describing, Instantiating and Evaluating a Reference Architecture: A Case Study. *J. Comput. Sci.* **2003**, *342*, 1–24.
36. Melnyk, P.; Djahel, S.; Nait-Abdesselam, F. Towards a Smart Parking Management System for Smart Cities. In Proceedings of the 2019 IEEE International Smart Cities Conference (ISC2), Casablanca, Morocco, 14–17 October 2019; pp. 542–546. [CrossRef]
37. Kubler, S.; Robert, J.; Hefnawy, A.; Cherifi, C.; Bouras, A.; Främling, K. IoT-based Smart Parking System for Sporting Event Management. In Proceedings of the 13th International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services, MOBIQUITOUS 2016, Hiroshima, Japan, 28 November–1 December 2016; pp. 104–114. [CrossRef]
38. Ajchariyavanich, C.; Limpisthira, T.; Chanjarasvichai, N.; Jareonwatanan, T.; Phongphanpanya, W.; Wareechuensuk, S.; Sricharee-onkul, S.; Tachatanitanont, S.; Ratanamahatana, C.; Prompoon, N.; et al. Park King: An IoT-based Smart Parking System. In Proceedings of the 2019 IEEE International Smart Cities Conference (ISC2), Casablanca, Morocco, 14–17 October 2019; pp. 729–734. [CrossRef]
39. Al-Kharusi, H.; Al-Bahadly, I. Intelligent parking management system based on image processing. *World J. Eng. Technol.* **2014**, *2*, 55–67. [CrossRef]
40. Bura, H.; Lin, N.; Kumar, N.; Malekar, S.; Nagaraj, S.; Liu, K. An Edge Based Smart Parking Solution Using Camera Networks and Deep Learning. In Proceedings of the 2018 IEEE International Conference on Cognitive Computing (ICCC), San Francisco, CA, USA, 2–7 July 2018; pp. 17–24. [CrossRef]
41. Perković, T.; Šolić, P.; Zargariasl, H.; Čoko, D.; Rodrigues, J.J. Smart Parking Sensors: State of the Art and Performance Evaluation. *J. Clean. Prod.* **2020**, *262*, 121181. [CrossRef]
42. Kang, K.; Cohen, S.; Hess, J.; Novak, W.; Peterson, A. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*; Technical Report CMU/SEI-90-TR-021; Software Engineering Institute: Pittsburgh, PA, USA, 1990.
43. Kruchten, P. The 4+1 View Model of Architecture. *IEEE Softw.* **1995**, *12*, 42–50. [CrossRef]
44. Hofmeister, C.; Nord, R.; Soni, D. *Applied Software Architecture*; Addison-Wesley Longman Publishing Co., Inc.: Boston, MA, USA, 1999.
45. Garland, J.; Anthony, R. *Large-Scale Software Architecture: A Practical Guide Using UML*, 1st ed.; Wiley Publishing: New York, NY, USA, 2002.
46. van Geest, M.; Tekinerdogan, B.; Catal, C. Design of a reference architecture for developing smart warehouses in industry 4.0. *Comput. Ind.* **2021**, *124*, 103343. [CrossRef]
47. Nakagawa, E.Y.; Oquendo, F.; Maldonado, J.C., Reference Architectures. In *Software Architecture 1*; John Wiley & Sons, Ltd.: Hoboken, NJ, USA, 2014; Chapter 2, pp. 55–82. [CrossRef]
48. Braun, G.A.; Estevez, E.; Fillottrani, P. A Reference Architecture for Ontology Engineering Web Environments. *J. Comput. Sci. Technol.* **2019**, *19*, e03. [CrossRef]
49. Richards, M. *Software Architecture Patterns*; O'Reilly Media, Inc.: Sebastopol, CA, USA, 2015.

50. Runeson, P.; Höst, M. Guidelines for conducting and reporting case study research in software engineering. *Empir. Softw. Eng.* **2009**, *14*, 131–164. [CrossRef]
51. Prieto-Diaz, R. Status report: Software reusability. *IEEE Softw.* **1993**, *10*, 61–66. [CrossRef]
52. Nakagawa, E.Y.; Maldonado, J.C. Reference architecture knowledge representation: An experience. In Proceedings of the 3rd International Workshop on Sharing and Reusing Architectural Knowledge, SHARK 2008, Leipzig, Germany, 13 May 2008; pp. 51–54. [CrossRef]
53. Guessi, M.; de Oliveira, L.B.R.; Nakagawa, E.Y. Representation of Reference Architectures: A Systematic Review. In Proceedings of the 23rd International Conference on Software Engineering & Knowledge Engineering (SEKE'2011), Eden Roc Renaissance, Miami Beach, FL, USA, 7–9 July 2011; Knowledge Systems Institute Graduate School: Skokie, IL, USA, 2011; pp. 782–785.
54. van Dinter, R.; Tekinerdogan, B.; Catal, C. Reference architecture for digital twin-based predictive maintenance systems. *Comput. Ind. Eng.* **2023**, *177*, 109099. [CrossRef]
55. Evans. *Domain-Driven Design: Tacking Complexity In the Heart of Software*; Addison-Wesley Longman Publishing Co., Inc.: Boston, MA, USA, 2003.
56. Bro, R.; Smilde, A.K. Principal component analysis. *Anal. Methods* **2014**, *6*, 2812–2831. [CrossRef]
57. Triantaphyllou, E., Multi-Criteria Decision Making Methods. In *Multi-Criteria Decision Making Methods: A Comparative Study*; Springer: Boston, MA, USA, 2000; pp. 5–21. [CrossRef]
58. Brown, S. Software Architecture for Developers: Volume 2—Visualise, Document and Explore Your Software Architecture; Leanpub, 2016. Available online: https://www.goodreads.com/book/show/33221619-software-architecture-for-developers (accessed on 10 September 2024).
59. Voirin, J.L. *Model-Based System and Architecture Engineering with the Arcadia Method*; Elsevier Ltd.: Oxford, UK, 2018. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.